

Reporte Técnico de Actividades Práctico-Experimentales Nro. 3

1. Datos de Identificación del Estudiante y la Práctica

Nombre de los estudiante(s)	Tyrone Encarnación Erick Andrade Jhoan Rojas Matías Ortega
Asignatura	Matemáticas Discretas
Ciclo	1A
Unidad	Unidad 3: Grafos y Árboles
Resultado de aprendizaje de la unidad	Entiende los principios del análisis combinatorio que resuelven problemas reales de su cotidianidad, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	003 FASE 1-2-3-4-5
Título de la Práctica	Teoría de Grafos, Algoritmos de Búsqueda, Árboles y Aplicaciones Computacionales
Nombre del Docente	Ing. Mario Enrique Cueva Hurtado
Fecha	Semana 9 y 10: del 1 al 12 de junio
Horario	Lunes, Martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	13 horas (3 horas por sesión APE)



UNL

Universidad
Nacional
de Loja

2. Objetivo(s) de la Práctica

- Comprender los fundamentos de la teoría de grafos: definición, terminología (vértices, aristas, grado, adyacencia, incidencia), tipos de grafos (dirigidos, no dirigidos, ponderados, completos, bipartitos) y representaciones (matriz de adyacencia, lista de adyacencia).
- Aplicar algoritmos fundamentales de búsqueda en grafos: BFS (Búsqueda a lo Ancho), DFS (Búsqueda en Profundidad) y Dijkstra (camino más corto), comprendiendo su funcionamiento paso a paso.
- Entender la teoría de árboles: definición, propiedades, tipos de árboles (binarios, BST, AVL), recorridos (inorden, preorden, posorden) y aplicaciones en ciencias de la computación.
- Utilizar software de modelado de grafos (Graph Online, Gephi) para representar, visualizar y analizar redes y estructuras de datos.
- Integrar los conocimientos de las tres unidades mediante un proyecto que aplique la lógica matemática, el álgebra de Boole y la teoría de grafos a un problema computacional real.

3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores
- Proyector multimedia
- Hojas de trabajo con ejercicios de teoría de grafos y árboles
- Guía paso a paso de los algoritmos BFS, DFS y Dijkstra
- Plantillas para construcción de grafos y árboles
- Investigación sobre aplicaciones de grafos en redes y optimización
- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL para entrega de evidencias
- Entorno virtual de aprendizaje (EVA-UNL)
- Laboratorio de computación
- Software Graph Online (<https://graphonline.ru/es>) o Gephi para modelado de grafos
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

4. Procedimiento / Metodología Ejecutada

FASE 1: Fundamentos de la Teoría de Grafos (Semana 13: 29 junio - 03 julio 2026) 1. Se realiza una presentación participativa de los fundamentos de la teoría de grafos: definición formal de grafo, terminología clave (vértice/nodo, arista/lado, lazo, bucle, grado de un vértice, camino, ciclo, clique), clasificación de grafos (dirigidos/no dirigidos, ponderados/no ponderados, simples/multigrafos, completos, bipartitos, regulares, conexos).

2. Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.

3. Práctica de representación de grafos: cada equipo representa el mismo grafo de tres formas diferentes: dibujo gráfico, matriz de adyacencia y lista de adyacencia. Comparan las ventajas y desventajas de cada representación en términos de eficiencia computacional y uso de memoria.

4. Los estudiantes investigan de forma autónoma sobre aplicaciones de grafos en redes de comunicaciones, optimización de rutas, bases de datos y redes sociales. Elaboran un miniinforme individual con al menos 3 aplicaciones reales documentadas.

5. Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean (Usar uno creado) un grafo representando una red de transporte en la ciudad de Loja, visualizándolo con el software y analizando sus propiedades (conectividad, caminos APE Nro. 003 — Unidad 3: Grafos y Árboles — 3 — mínimos, nodos críticos). Horario de tutorías: lunes 15h30 a 17h30 (presencial, Edificio 8, Sala A.8.2.1). Devolución de calificaciones: máximo 5 días hábiles.

FASE 2: Algoritmos de Búsqueda y Caminos Mínimos (Semana 14: 06-10 julio 2026)

1. El docente explica mediante Flipped Classroom los algoritmos fundamentales para grafos: BFS (Breadth-First Search / Búsqueda a lo Ancho), DFS (Depth-First Search / Búsqueda en Profundidad) y Dijkstra (Camino Más Corto), detallando su funcionamiento paso a paso, complejidad temporal y aplicaciones específicas.

2. Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.



3. Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.
4. Presentación grupal de resultados: cada equipo expone su análisis de optimización, explicando cómo los algoritmos de grafos pueden resolver problemas reales (rutas de entrega, diseño de redes, navegación GPS).
5. Discusión sobre la equidad digital y la importancia de la optimización de redes de comunicación para reducir la brecha digital en comunidades rurales del Ecuador. Reflexión sobre la responsabilidad ética en el diseño de algoritmos.

5. Resultados

5.1 FASE 1:

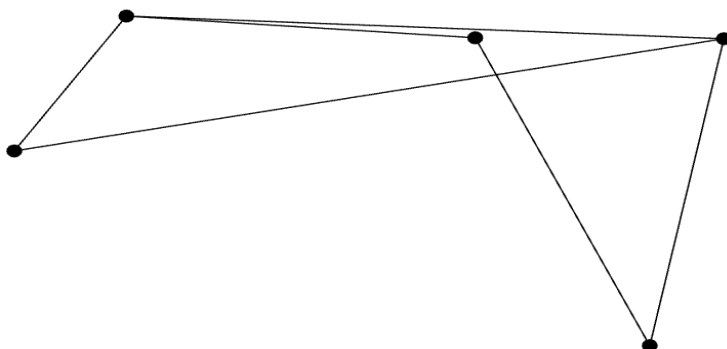
- a. Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.

Ejercicio 1. Red de amigos

Enunciado: Ana es amiga de Bruno y Carla. Bruno es amigo de Diego. Carla es amiga de Diego y Elena. Elena conoce a Ana, pero no tiene amistad con Bruno ni con Diego.

Elemento	Respuesta
Tipo de grafo	No dirigido, porque la amistad/conocimiento se representa como relación mutua entre dos personas.
Vértices (V)	{Ana, Bruno, Carla, Diego, Elena}
Aristas (E)	{(Ana, Bruno), (Ana, Carla), (Bruno, Diego), (Carla, Diego), (Carla, Elena), (Elena, Ana)}
Grados	Ana = 3; Bruno = 2; Carla = 3; Diego = 2; Elena = 2.
Propiedades	Simple, conexo, no completo y cíclico. Tiene ciclos, por ejemplo Ana - Bruno - Diego - Carla - Ana.

Grafo en Gephi:

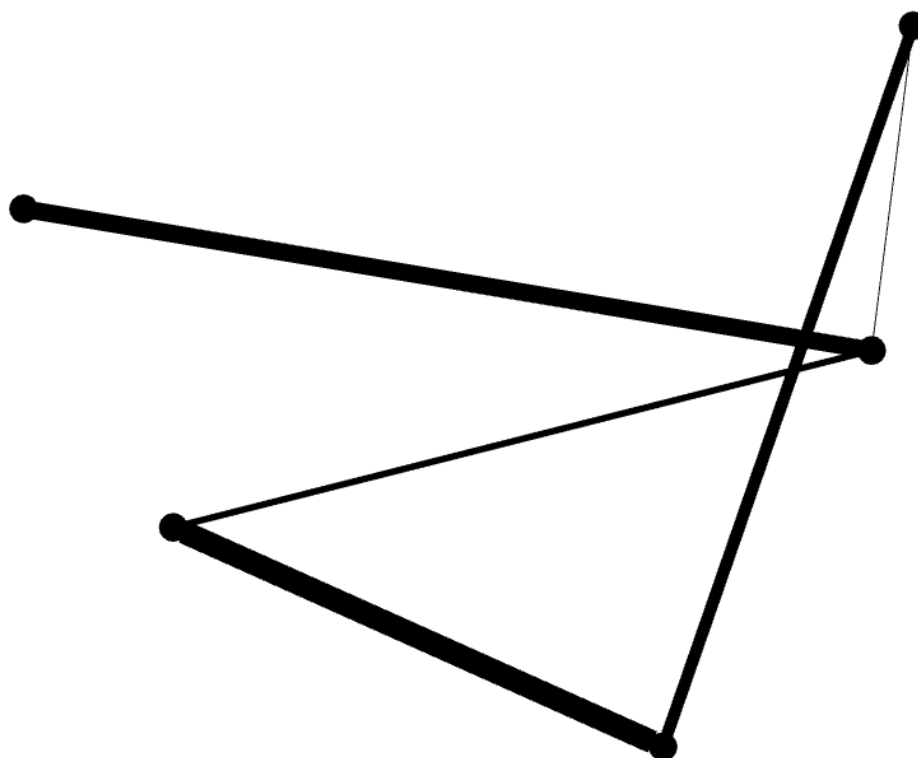


Ejercicio 2. Red de transporte urbano

Enunciado: Paradas A, B, C, D y E. Rutas: A-B, A-C, B-D, C-D y D-E. Tiempos: A-B 8 min, A-C 6 min, B-D 5 min, C-D 4 min y D-E 7 min.

Elemento	Respuesta
Tipo de grafo	No dirigido y ponderado, porque las rutas funcionan en ambos sentidos y cada arista tiene un tiempo.
Vértices (V)	{A, B, C, D, E}
Aristas ponderadas (E)	{(A, B, 8), (A, C, 6), (B, D, 5), (C, D, 4), (D, E, 7)}
Grados	A = 2; B = 2; C = 2; D = 3; E = 1.
Propiedades	Conexo, no completo y cíclico. Tiene el ciclo A - B - D - C - A. No es árbol porque contiene un ciclo.

Grafo en Gephi:

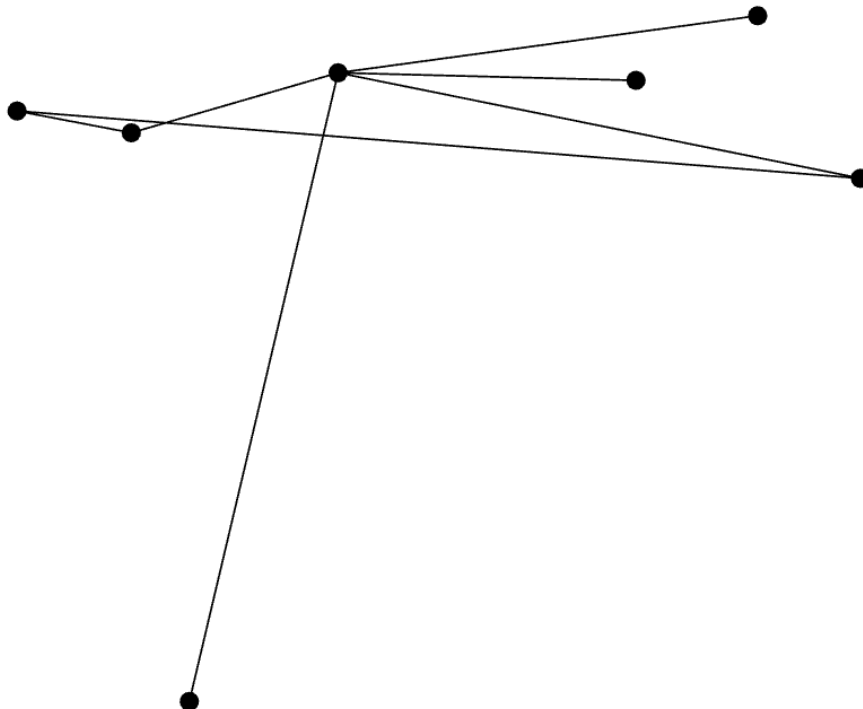


Ejercicio 3. Topología de red de computadoras

Enunciado: El router está conectado al servidor y al switch principal. El switch principal está conectado a PC1, PC2, PC3 y a una impresora. El servidor también está conectado directamente a la impresora.

Elemento	Respuesta
Tipo de grafo	No dirigido, porque las conexiones físicas permiten comunicación en ambos sentidos.
Vértices (V)	{Router, Servidor, Switch, PC1, PC2, PC3, Impresora}
Aristas (E)	{(Router, Servidor), (Router, Switch), (Switch, PC1), (Switch, PC2), (Switch, PC3), (Switch, Impresora), (Servidor, Impresora)}
Grados	Router = 2; Servidor = 2; Switch = 5; PC1 = 1; PC2 = 1; PC3 = 1; Impresora = 2.
Propiedades	Conexo, no completo y cíclico. Tiene el ciclo Router - Servidor - Impresora - Switch - Router. No es árbol porque tiene ciclo.

Grafo en Gephi:



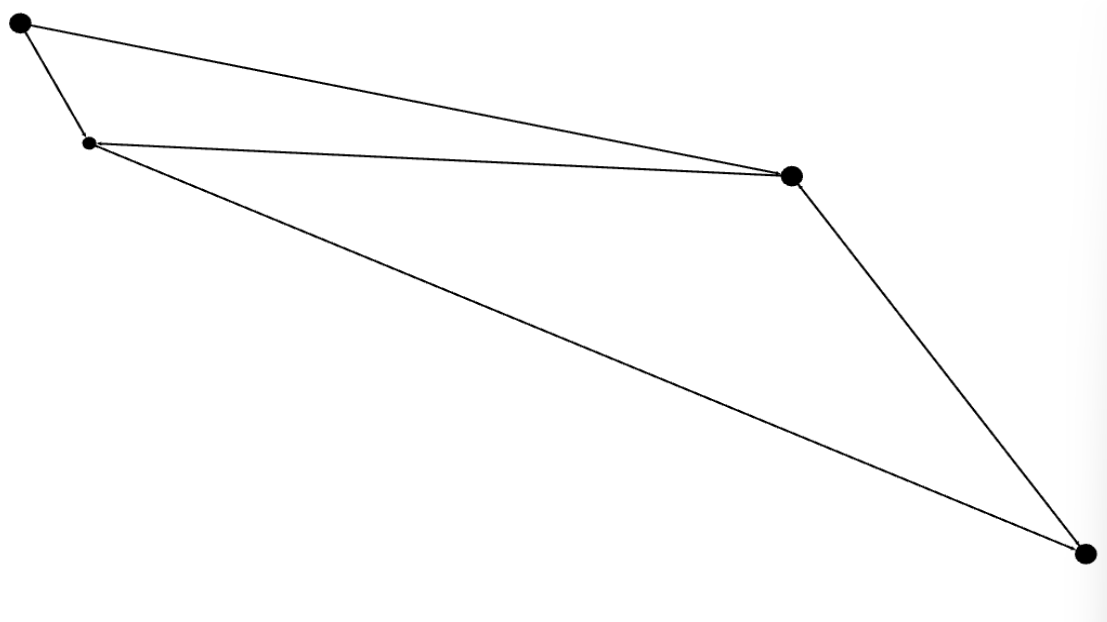


Ejercicio 4. Seguidores en una red social

Enunciado: Lucía sigue a Mateo y a Sofía. Mateo sigue a Sofía. Sofía sigue a Lucía. Andrés sigue a Lucía y a Mateo. Nadie sigue a Andrés.

Elemento	Respuesta
Tipo de grafo	Dirigido, porque seguir a una persona no implica que esa persona también siga de vuelta.
Vértices (V)	{Lucía, Mateo, Sofía, Andrés}
Aristas dirigidas (E)	{Lucía -> Mateo, Lucía -> Sofía, Mateo -> Sofía, Sofía -> Lucía, Andrés -> Lucía, Andrés -> Mateo}
Grado de entrada	Lucía = 2; Mateo = 2; Sofía = 2; Andrés = 0.
Grado de salida	Lucía = 2; Mateo = 1; Sofía = 1; Andrés = 2.
Propiedades	Dirigido, no completo y con ciclos dirigidos. Ejemplos: Lucía -> Sofía -> Lucía y Lucía -> Mateo -> Sofía -> Lucía.

Grafo en Gephi:

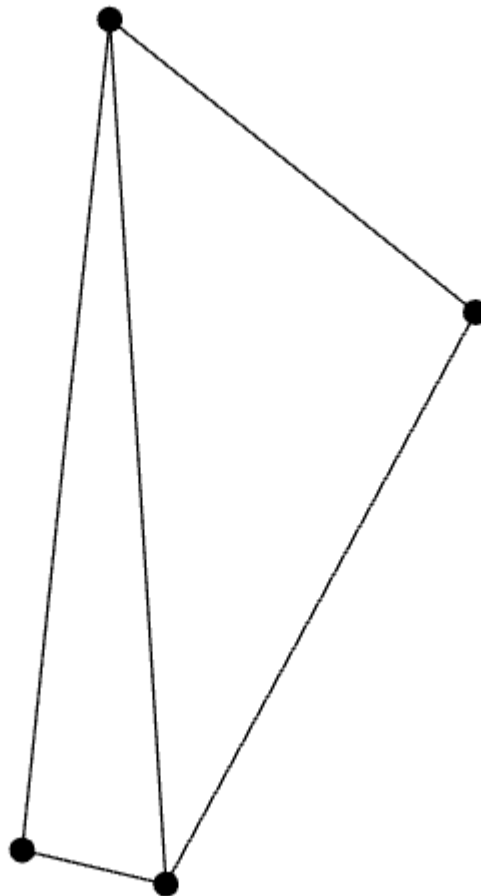


Ejercicio 5. Red de colaboración escolar

Enunciado: Juan trabaja con María, María trabaja con Pedro, Pedro trabaja con Ana, Ana trabaja con Juan y Juan también trabaja con Pedro.

Elemento	Respuesta
Tipo de grafo	No dirigido, porque trabajar con alguien se interpreta como colaboración mutua.
Vértices (V)	{Juan, María, Pedro, Ana}
Aristas (E)	{(Juan, María), (María, Pedro), (Pedro, Ana), (Ana, Juan), (Juan, Pedro)}
Grados	Juan = 3; María = 2; Pedro = 3; Ana = 2.
Propiedades	Simple, conexo, no completo y cíclico. Tiene ciclos como Juan - María - Pedro - Juan y Juan - Pedro - Ana - Juan.

Grafo en Gephi:



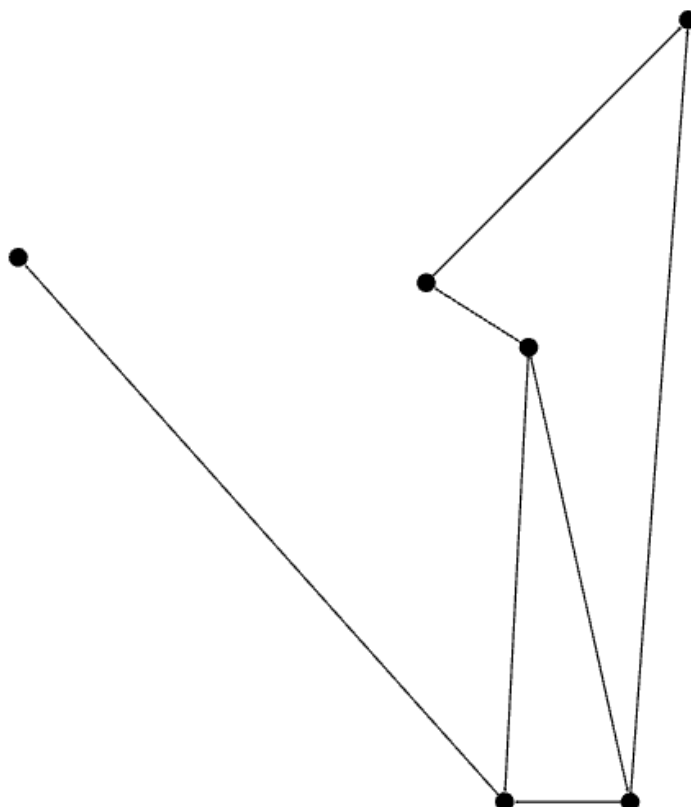


Ejercicio 6. Rutas de entrega de paquetes

Enunciado: Desde el almacén central M se envían paquetes hacia Norte, Sur y Este. Desde Norte se envía a Oeste. Desde Este se envía a Oeste y Sur. Desde Sur se envía a Centro.

Elemento	Respuesta
Tipo de grafo	Dirigido, porque las rutas de compl tienen una dirección específica.
Vértices (V)	{M, Norte, Sur, Este, Oeste, Centro}
Aristas dirigidas €	{M -> Norte, M -> Sur, M -> Este, Norte -> Oeste, Este -> Oeste, Este -> Sur, Sur -> Centro}
Grado de entrada	M = 0; Norte = 1; Sur = 2; Este = 1; Oeste = 2; Centro = 1.
Grado de salida	M = 3; Norte = 1; Sur = 1; Este = 2; Oeste = 0; Centro = 0.
Propiedades	Todas las zonas son alcanzables desde M. No hay ciclo dirigido. No es complete.

Grafo en Gephi:

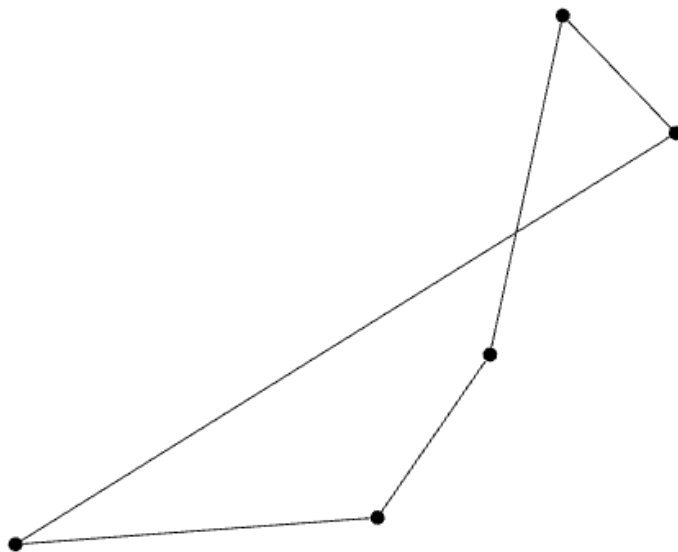


Ejercicio 7. Prerrequisitos de materias

Enunciado: Programación I es requisito de Programación II y Bases de Datos. Programación II es requisito de Estructuras de Datos. Estructuras de Datos y Bases de Datos son requisitos de Proyecto Integrador.

Elemento	Respuesta
Tipo de grafo	Dirigido, usando $A \rightarrow B$ cuando A es requisito para cursar B.
Vértices (V)	{Programación I, Programación II, Estructuras de Datos, Bases de Datos, Proyecto Integrador}
Aristas dirigidas (E)	{ Programación I $\rightarrow$ Programación II, Programación II $\rightarrow$ Estructuras de Datos , Programación I $\rightarrow$ Bases de Datos, Estructuras de Datos $\rightarrow$ Proyecto Integrador, Bases de Datos $\rightarrow$ Proyecto Integrador}
Grado de entrada	Programación I = 0; Programación II = 1; Estructuras de Datos = 1; Bases de Datos = 1; Proyecto Integrador = 2.
Grado de salida	Programación I = 2; Programación II = 1; Estructuras de Datos = 1; Bases de Datos = 1; Proyecto Integrador = 0.
Propiedades	Dirigido y acíclico. Es acíclico porque ninguna materia vuelve a ser requisito de una materia anterior.

Grafo en Gephi:





UNL

Universidad
Nacional
de Loja

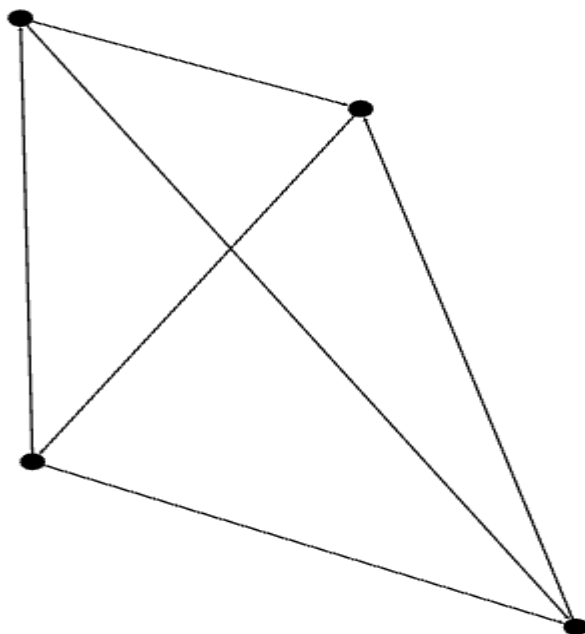
FEIRNNR - Carrera de Computación

Ejercicio 8. Red de tuberías de agua

Enunciado: El tanque T abastece a A y B. A se conecta con C. B se conecta con C y D. C se conecta con E. D también se conecta con E.

Elemento	Respuesta
Tipo de grafo	No dirigido, porque se modelan conexiones de tuberías entre nodos.
Vértices (V)	{T, A, B, C, D, E}
Aristas $\in$	{(T, A), (T, B), (A, C), (B, C), (B, D), (C, E), (D, E)}
Grados	T = 2; A = 2; B = 3; C = 3; D = 2; E = 2.
Propiedades	Conexo, no completo y cíclico. Sí existe más de un camino entre T y E, por ejemplo T – A – C – E y T – B – D – E.

Grafo en Gephi:

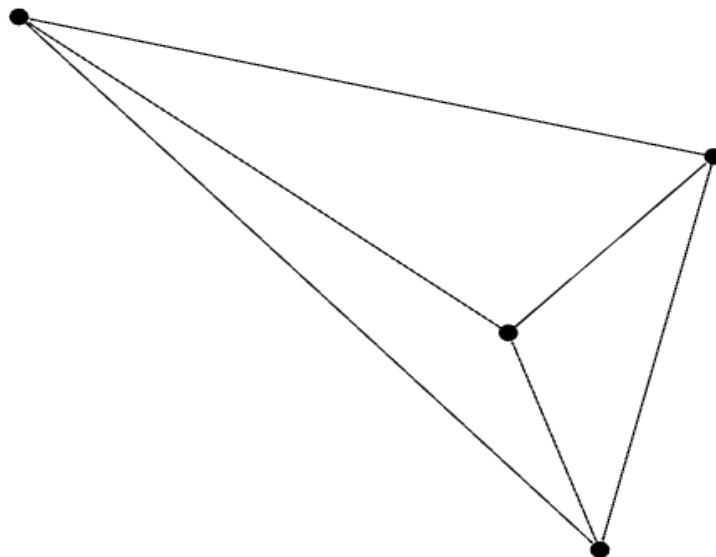


Ejercicio 9. Torneo deportivo

Enunciado: A venció a B y C. B venció a D. C venció a B y D. D venció a A.

Elemento	Respuesta
Tipo de grafo	Dirigido, porque $X \rightarrow Y$ significa que X venció a Y.
Vértices (V)	{A, B, C, D}
Aristas dirigidas (E)	{A $\rightarrow$ B, A $\rightarrow$ C, B $\rightarrow$ D, C $\rightarrow$ B, C $\rightarrow$ D, D $\rightarrow$ A}
Grado de entrada	A = 1; B = 2; C = 1; D = 2.
Grado de salida	A = 2; B = 1; C = 2; D = 1.
Propiedades	Dirigido, no completo y cíclico. Hay ciclos dirigidos, por ejemplo A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ A y A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ A.

Grafo en Gephi:

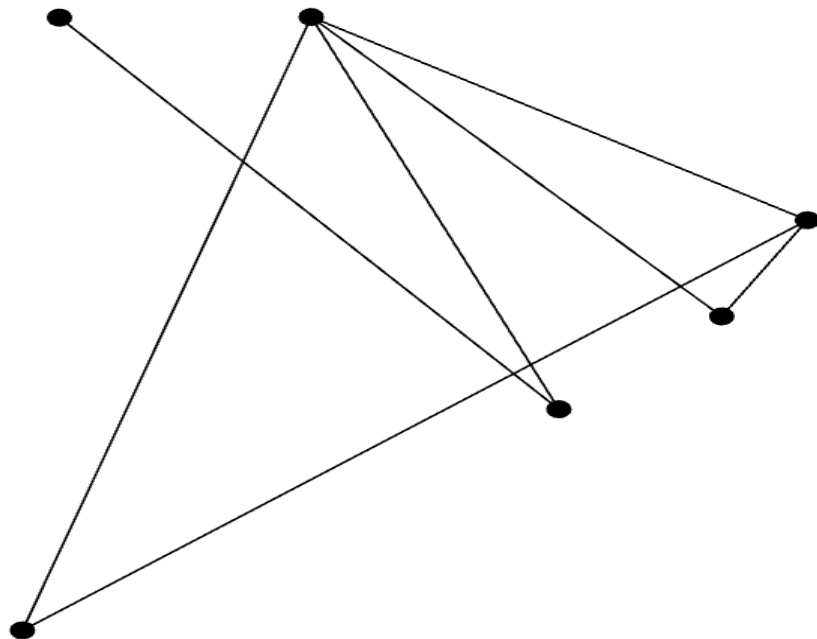


Ejercicio 10. Comunicación entre oficinas

Enunciado: Central se comunica con Finanzas, Ventas y Soporte. Finanzas con Recursos Humanos. Ventas con Soporte. Soporte con Sistemas. Sistemas con Central. La comunicación es bidireccional.

Elemento	Respuesta
Tipo de grafo	No dirigido, porque la comunicación es bidireccional.
Vértices (V)	{Central, Finanzas, Ventas, Soporte, Recursos Humanos, Sistemas}
Aristas (E)	{(Central, Finanzas), (Central, Ventas), (Central, Soporte), (Finanzas, Recursos Humanos), (Ventas, Soporte), (Soporte, Sistemas), (Sistemas, Central)}
Grados	Central = 4; Finanzas = 2; Ventas = 2; Soporte = 3; Recursos Humanos = 1; Sistemas = 2.
Propiedades	Conexo, no completo y cíclico. No es árbol porque tiene ciclos, por ejemplo Central - Ventas - Soporte - Central.

Grafo en Gephi:



- b. Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean (Usar uno creado) un grafo representando una red de transporte en la ciudad de Loja, visualizándolo con el software y analizando sus propiedades (conectividad, caminos mínimos, nodos críticos).**

Descripción del Grafo:

Grafo Vial de la Provincia de Loja, Ecuador

Para la elaboración de este punto del informe de desarrollo un grafo vial construido para representar la red de conexión entre los cantones de la provincia de Loja, Ecuador. Cada cantón (o su cabecera cantonal) se modela como un nodo, y cada carretera real o simulada entre dos cantones se modela como una arista no dirigida cuyo peso corresponde a la distancia aproximada en kilómetros.

El grafo resultante tiene 17 nodos y 22 aristas, y sirve de base para el simulador interactivo de rutas (cálculo de la ruta más corta y la ruta más larga entre dos cantones, con animación del recorrido).

Descripción de los datos:

Nodos (cantones)

Cantón
Loja
Catamayo
Saraguro
Cariamanga (Calvas)
Catacocha (Paltas)



Cantón
Macará
Amaluza (Espíndola)
Alamor (Puyango)
Celica
Gonzanamá
Zapotillo
Sozoranga
Pindal
Chaguarpamba
Olmedo
Vilcabamba
Quilanga

Aristas (vías)

Origen	Destino	Distancia (km)	Tipo de vía
Loja	Catamayo	35	Primaria
Loja	Saraguro	64	Primaria
Loja	Vilcabamba	42	Secundaria
Loja	Gonzanamá	65	Secundaria



Origen	Destino	Distancia (km)	Tipo de vía
Catamayo	Catacocha	60	Secundaria
Catamayo	Chaguarpamba	45	Terciaria
Chaguarpamba	Olmedo	25	Terciaria
Catacocha	Olmedo	20	Terciaria
Gonzanamá	Cariamanga	40	Secundaria
Gonzanamá	Quilanga	20	Terciaria
Vilcabamba	Quilanga	45	Terciaria
Quilanga	Cariamanga	30	Terciaria
Cariamanga	Macará	75	Primaria
Cariamanga	Amaluza	55	Secundaria
Cariamanga	Sozoranga	35	Secundaria
Sozoranga	Macará	50	Terciaria
Catacocha	Celica	45	Secundaria
Celica	Alamor	35	Secundaria
Celica	Pindal	25	Terciaria
Alamor	Zapotillo	55	Terciaria
Pindal	Zapotillo	45	Terciaria
Zapotillo	Macará	80	Secundaria (vía costera, sector Velacruz)

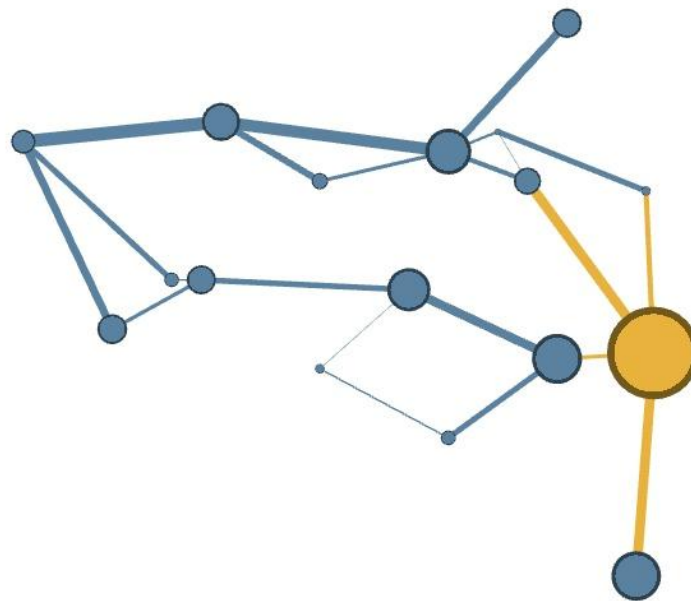
Análisis de conectividad

¿El grafo es conexo?: Sí. Existe al menos un camino entre cualquier par de cantones.

Conectividad de nodos (κ): 1 — basta con que falle un único nodo (cantón) para que el grafo se desconecte en componentes separadas.

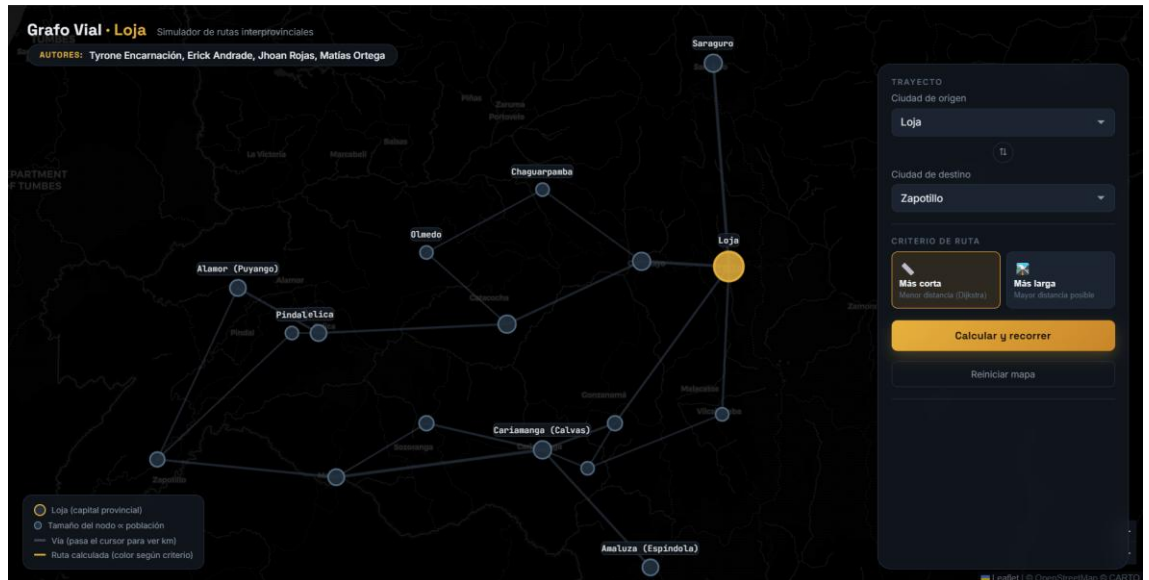
Conectividad de aristas (λ): 1 — basta con que falle una única vía para desconectar la red.

Modelado del Grafo en Gephi:

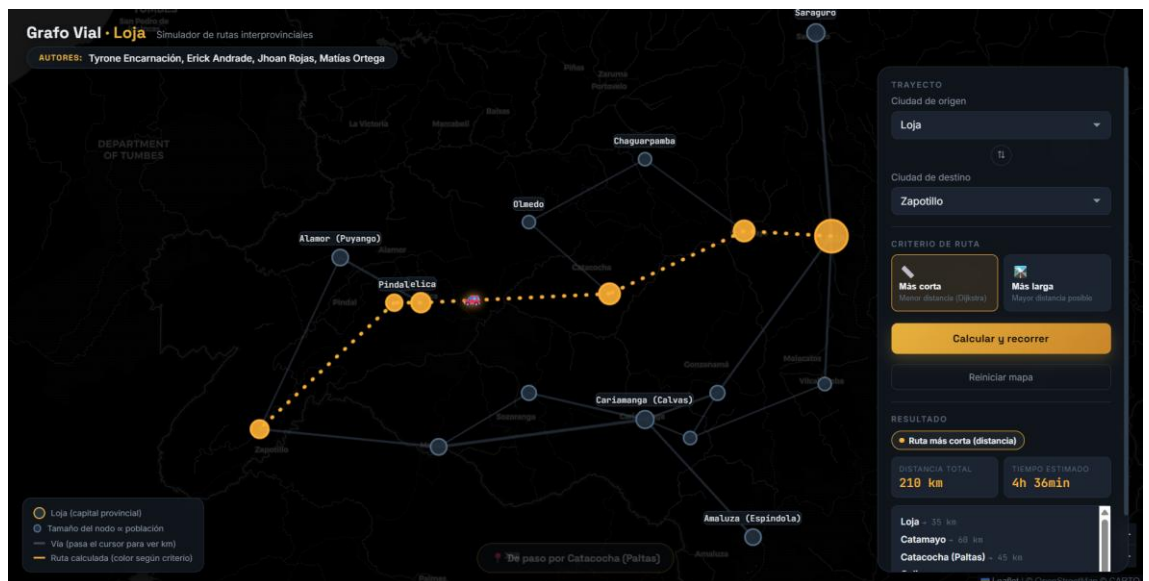


Implementación del Grafo en una página web:

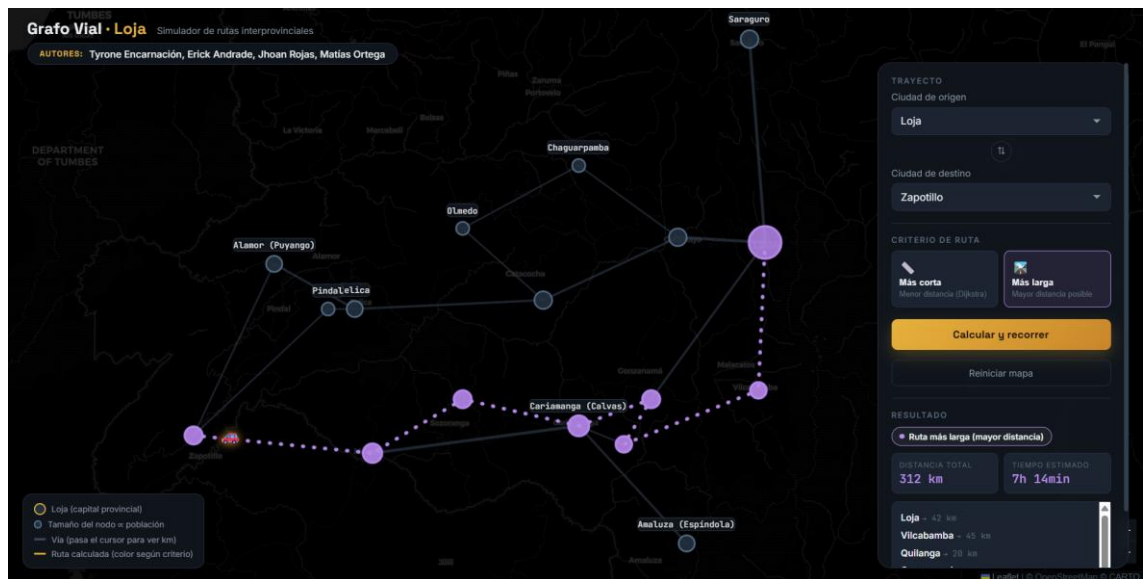
Grafo no dirigido



Grafo no dirigido (Calcular ruta más corta desde Loja hasta Zapotillo)



Ejemplo del grafo (Calcular ruta más larga desde Loja hasta Zapotillo)



Link de la página web:

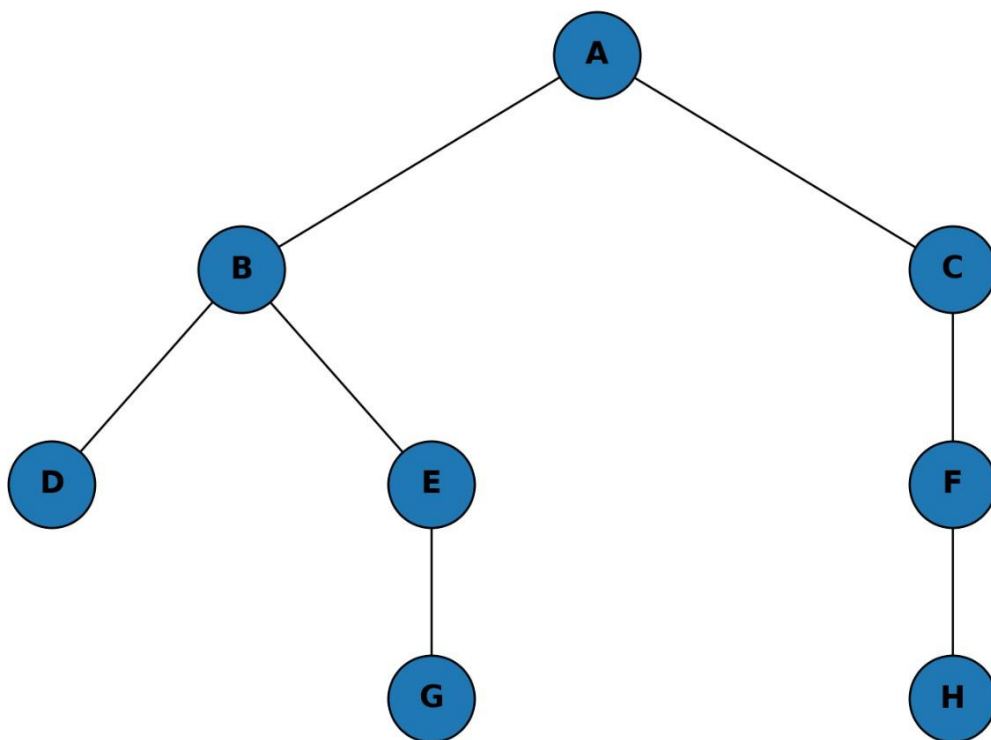
<https://verdant-dragon-54e351.netlify.app/>

5.3 FASE 2:

- a. Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.

Algoritmo BFS

Grafo no dirigido para BFS - Inicio: A





Inicio: A

Orden BFS:

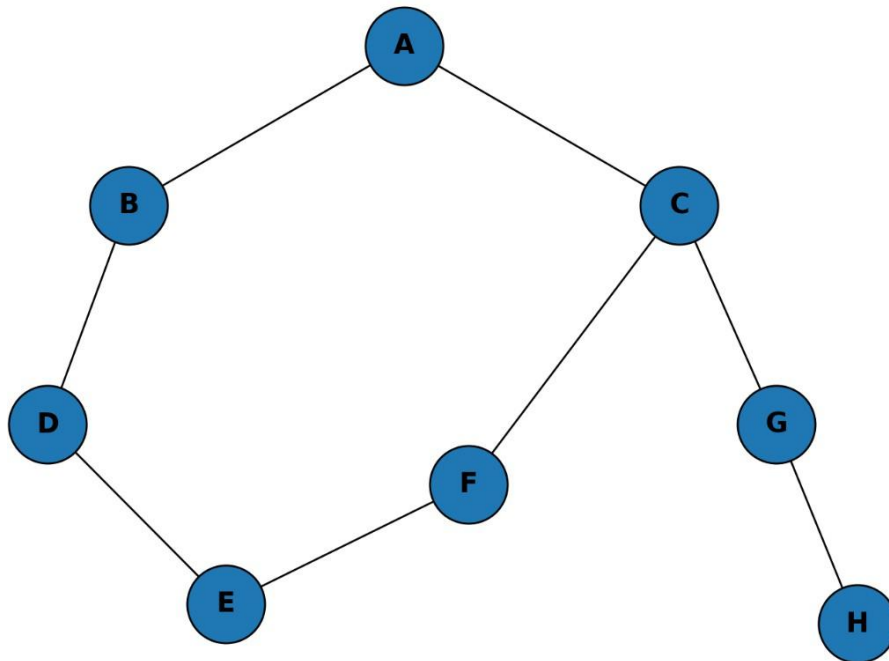
A → B → C → D → E → F → G → H

Cola paso a paso:

Paso	Nodo visitado	Cola
1	A	B, C
2	B	C, D, E
3	C	D, E, F
4	D	E, F
5	E	F, G
6	F	G, H
7	G	H
8	H	vacía

Algoritmo DFS

Grafo no dirigido para DFS - Inicio: A



Inicio: A

Orden DFS:

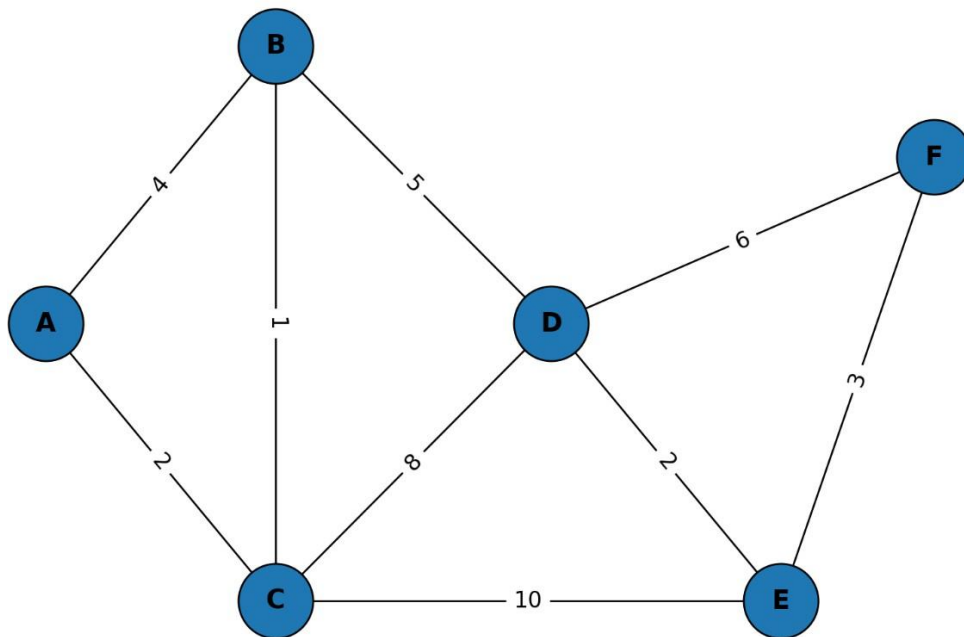
A → B → D → E → F → C → G → H

Pila / recorrido:

Paso	Nodo visitado	Acción
1	A	va a B
2	B	va a D
3	D	va a E
4	E	va a F
5	F	va a C
6	C	va a G
7	G	va a H
8	H	termina

Algoritmo Dijkstra

Grafo ponderado para Dijkstra - Inicio: A



Inicio: A

Distancias finales desde A:

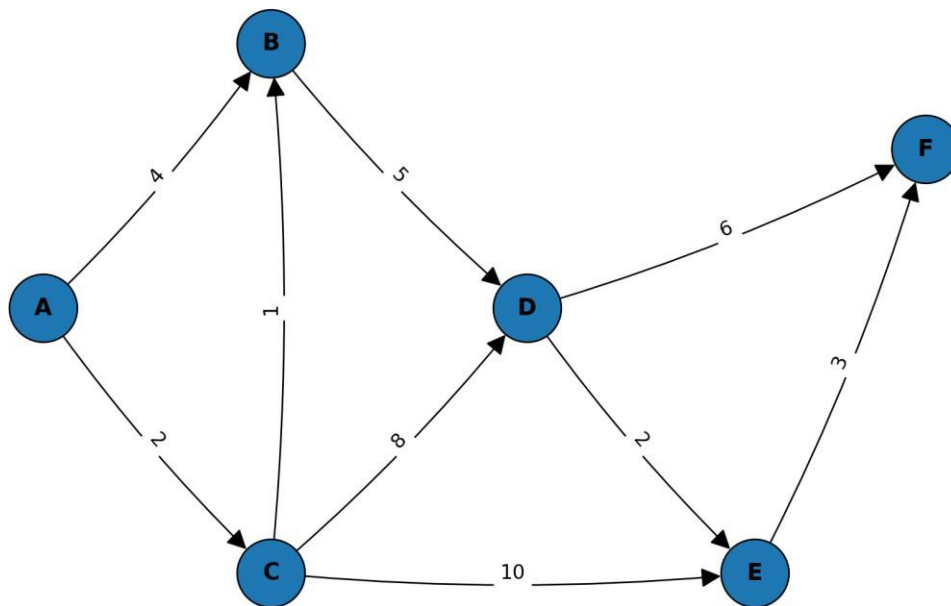
Nodo	Distancia mínima	Camino
A	0	A
C	2	A → C
B	3	A → C → B
D	8	A → C → B → D
E	10	A → C → B → D → E
F	13	A → C → B → D → E → F

Camino más corto de A a F:

A → C → B → D → E → F = 13

Algoritmo Dijkstra

Grafo ponderado dirigido para Dijkstra - Nodo inicial: A



Dijkstra paso a paso

Paso	Nodo fijo	Distancias acumuladas
1	A	A=0, B=4, C=2, D=∞, E=∞, F=∞
2	C	A=0, B=3, C=2, D=10, E=12, F=∞
3	B	A=0, B=3, C=2, D=8, E=12, F=∞
4	D	A=0, B=3, C=2, D=8, E=10, F=14
5	E	A=0, B=3, C=2, D=8, E=10, F=13
6	F	A=0, B=3, C=2, D=8, E=10, F=13

Orden de descubrimiento / fijación

A → C → B → D → E → F





Caminos resultantes

Nodo destino	Distancia mínima	Camino
A	0	A
C	2	A → C
B	3	A → C → B
D	8	A → C → B → D
E	10	A → C → B → D → E
F	13	A → C → B → D → E → F

Camino más corto de **A a F**:

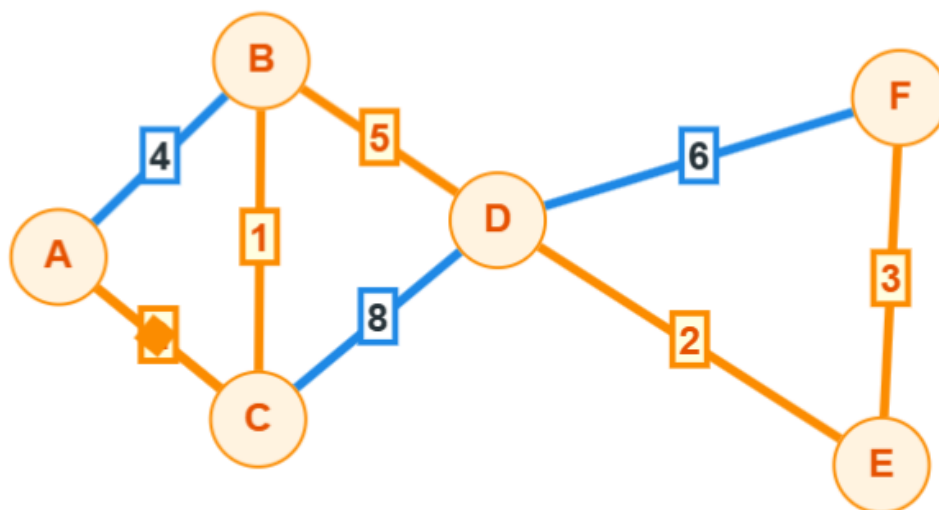
A → C → B → D → E → F = 13



- b. Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.

GRAFO PONDERADO PARA DIJKSTRA – INICIO A – FINAL F

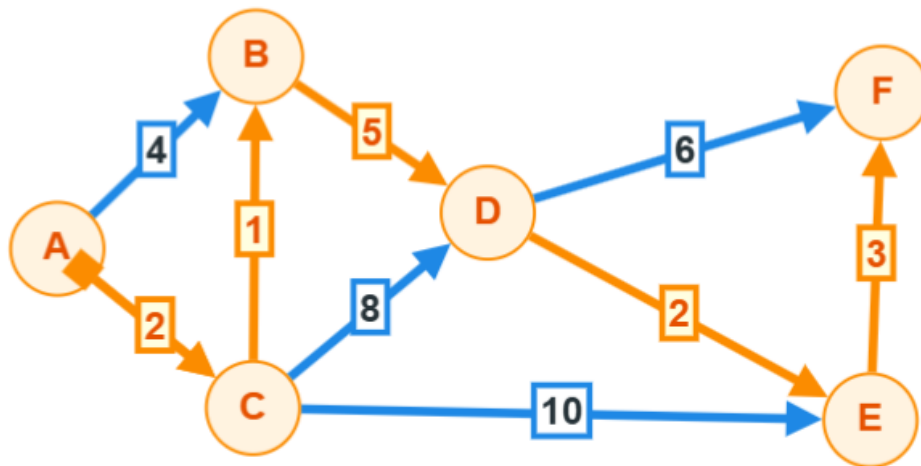
La longitud del camino más corto es 13: $A \Rightarrow C \Rightarrow B \Rightarrow D \Rightarrow E \Rightarrow F$



Conclusión: El camino más corto obtenido usando el algoritmo de Dijkstra es 13 por ende coincide con los obtenidos manualmente.

GRAFO PONDERADO DIRIGIDO PARA DIJKSTRA – INICIO A – FINAL F

La longitud del camino más corto es 13: $A \Rightarrow C \Rightarrow B \Rightarrow D \Rightarrow E \Rightarrow F$



Conclusión: El camino más corto obtenido usando el algoritmo de Dijkstra es 13 por ende coincide con los obtenidos manualmente.

FASE 3: Árboles y Estructuras de Datos Arbóreas (Semana 15: 13-17 julio 2026)

a. Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.

Introducción

A continuación, se construyen tres BST distintos a partir de conjuntos de valores numéricos diferentes. Cada árbol se documenta insertando un valor a la vez y mostrando el gráfico del árbol resultante después de cada inserción. Al final de cada árbol se presentan los tres recorridos clásicos: inorden, preorden y postorden.

Reglas de inserción utilizadas

- Si el árbol está vacío, el valor insertado se convierte en la raíz.
- Si el valor es menor que el nodo actual, se compara con el subárbol izquierdo.
- Si el valor es mayor que el nodo actual, se compara con el subárbol derecho.
- El proceso se repite hasta encontrar una posición vacía (hoja) donde insertar el nuevo nodo.

Definición de los recorridos

- Inorden (Izquierda – Raíz – Derecha): en un BST produce los valores en orden ascendente.
- Preorden (Raíz – Izquierda – Derecha): útil para copiar o serializar el árbol.
- Postorden (Izquierda – Derecha – Raíz): útil para eliminar el árbol o evaluar expresiones.

Árbol Binario de Búsqueda N.º 1

Se insertará la siguiente secuencia de valores, uno a la vez:

50, 30, 70, 20, 40, 60, 80

Paso 1 — Insertar 50

Es el primer valor, se convierte en la raíz.

50

Paso 2 — Insertar 30

$30 < 50 \rightarrow$ va a la izquierda de 50.

50

/

30



unl

Universidad
Nacional
de Loja

Paso 3 — Insertar 70

$70 > 50 \rightarrow$ va a la derecha de 50.

```
  50_
 /  \
30  70
```

Paso 4 — Insertar 20

$20 < 50 \rightarrow$ izquierda; $20 < 30 \rightarrow$ izquierda de 30.

```
  50_
 /  \
30  70
 /
20
```

Paso 5 — Insertar 40

$40 < 50 \rightarrow$ izquierda; $40 > 30 \rightarrow$ derecha de 30.

```
  50_
 /  \
30_  70
 /  \
20  40
```

Paso 6 — Insertar 60

$60 > 50 \rightarrow$ derecha; $60 < 70 \rightarrow$ izquierda de 70.

```
  50_
 /  \
30_  70
 /  \ /
20 40 60
```

Paso 7 — Insertar 80

$80 > 50 \rightarrow$ derecha; $80 > 70 \rightarrow$ derecha de 70.

```
  50_
 /  \
30_  70_
 /  \ /  \
20 40 60 80
```



Recorridos del Árbol N.º 1

Recorrido	Orden de visita de nodos
Inorden (I-R-D)	20 - 30 - 40 - 50 - 60 - 70 - 80
Preorden (R-I-D)	50 - 30 - 20 - 40 - 70 - 60 - 80
Postorden (I-D-R)	20 - 40 - 30 - 60 - 80 - 70 - 50

Árbol Binario de Búsqueda N.º 2

Se insertará la siguiente secuencia de valores, uno a la vez:

15, 10, 20, 8, 12, 17, 25, 6

Paso 1 — Insertar 15

Es el primer valor, se convierte en la raíz.

15

Paso 2 — Insertar 10

$10 < 15 \rightarrow$ va a la izquierda de 15.

15

/

10

Paso 3 — Insertar 20

$20 > 15 \rightarrow$ va a la derecha de 15.

15_

/ \

10 20

Paso 4 — Insertar 8

$8 < 15 \rightarrow$ izquierda; $8 < 10 \rightarrow$ izquierda de 10.

15_

/ \

10 20

/

8



unl

Universidad
Nacional
de Loja

Paso 5 — Insertar 12

$12 < 15 \rightarrow$ izquierda; $12 > 10 \rightarrow$ derecha de 10.

```
  _15_
 /  \
10_ 20
 /  \
8 12
```

Paso 6 — Insertar 17

$17 > 15 \rightarrow$ derecha; $17 < 20 \rightarrow$ izquierda de 20.

```
  _15_
 /  \
10_ 20
 / \ /
8 12 17
```

Paso 7 — Insertar 25

$25 > 15 \rightarrow$ derecha; $25 > 20 \rightarrow$ derecha de 20.

```
  _15_
 /  \
10_ 20_
 / \ / \
8 12 17 25
```

Paso 8 — Insertar 6

$6 < 15 \rightarrow$ izquierda; $6 < 10 \rightarrow$ izquierda; $6 < 8 \rightarrow$ izquierda de 8.

```
  _15_
 /  \
10_ 20_
 / \ / \
8 12 17 25
/
```




Recorridos del Árbol N.º 2

Recorrido	Orden de visita de nodos
Inorden (I-R-D)	6 - 8 - 10 - 12 - 15 - 17 - 20 - 25
Preorden (R-I-D)	15 - 10 - 8 - 6 - 12 - 20 - 17 - 25
Postorden (I-D-R)	6 - 8 - 12 - 10 - 17 - 25 - 20 - 15

Árbol Binario de Búsqueda N.º 3

Se insertará la siguiente secuencia de valores, uno a la vez:

45, 25, 65, 10, 35, 55, 75, 5, 15, 30, 40

Paso 1 — Insertar 45

Es el primer valor, se convierte en la raíz.

45

Paso 2 — Insertar 25

$25 < 45 \rightarrow$ va a la izquierda de 45.

45

/

25

Paso 3 — Insertar 65

$65 > 45 \rightarrow$ va a la derecha de 45.

45_

/ \

25 65

Paso 4 — Insertar 10

$10 < 45 \rightarrow$ izquierda; $10 < 25 \rightarrow$ izquierda de 25.

45_

/ \

25 65

/

10



Paso 5 — Insertar 35

$35 < 45 \rightarrow$ izquierda; $35 > 25 \rightarrow$ derecha de 25.

```
  45_
 /   \
25_ 65
 /   \
10 35
```

Paso 6 — Insertar 55

$55 > 45 \rightarrow$ derecha; $55 < 65 \rightarrow$ izquierda de 65.

```
  45_
 /   \
25_ 65
 / \ /
10 35 55
```

Paso 7 — Insertar 75

$75 > 45 \rightarrow$ derecha; $75 > 65 \rightarrow$ derecha de 65.

```
  45_
 /   \
25_ 65_
 / \ / \
10 35 55 75
```

Paso 8 — Insertar 5

$5 < 45 \rightarrow$ izquierda; $5 < 25 \rightarrow$ izquierda; $5 < 10 \rightarrow$ izquierda de 10.

```
  45_
 /   \
25_ 65_
 / \ / \
10 35 55 75
/
5
```

Paso 9 — Insertar 15

$15 < 45 \rightarrow$ izquierda; $15 < 25 \rightarrow$ izquierda; $15 > 10 \rightarrow$ derecha de 10.

```
  45_
 /   \
```



```
    25_  65_  
  /  \ /  \  
10_ 35 55 75  
/  \  
5 15
```

Paso 10 — Insertar 30

$30 < 45 \rightarrow$ izquierda; $30 > 25 \rightarrow$ derecha; $30 < 35 \rightarrow$ izquierda de 35.

```
      45_____  
    /      \  
   25____  65_  
 /  \ /  \  
10_ 35 55 75  
/  \ /  
5 15 30
```

Paso 11 — Insertar 40

$40 < 45 \rightarrow$ izquierda; $40 > 25 \rightarrow$ derecha; $40 > 35 \rightarrow$ derecha de 35.

```
      45_____  
    /      \  
   25____  65_  
 /  \ /  \  
10_ 35_ 55 75  
/  \ /  \  
5 15 30 40
```

Recorridos del Árbol N.º 3

Recorrido	Orden de visita de nodos
Inorden (I-R-D)	5 - 10 - 15 - 25 - 30 - 35 - 40 - 45 - 55 - 65 - 75
Preorden (R-I-D)	45 - 25 - 10 - 5 - 15 - 35 - 30 - 40 - 65 - 55 - 75
Postorden (I-D-R)	5 - 15 - 10 - 30 - 40 - 35 - 25 - 55 - 75 - 65 - 45

Conclusiones

- En los tres árboles, el recorrido inorden produjo siempre los valores en orden ascendente, lo cual verifica que la propiedad fundamental del BST se respetó durante cada inserción.
- El orden de inserción determina la forma final del árbol: el Árbol N.º 2 mostró una rama más profunda por insertar valores de forma más secuencial, mientras que el Árbol N.º 3 resultó más equilibrado.
- El recorrido preorden resulta útil para reconstruir el árbol o copiarlo, ya que conserva el orden en que se insertó cada nodo respecto a su posición como raíz de cada subárbol.
- El recorrido postorden es apropiado para operaciones de eliminación, pues garantiza que los hijos se procesen antes que su nodo padre.

b. Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión). Elaboran un miniinforme con exposición apoyada en recursos TIC.

Teoría de grafos/árboles y las estructuras de datos utilizadas en programación

1. Introducción

Un grafo es un par (V, E) de vértices y aristas; un árbol es un caso particular: un grafo acíclico y conexo con una raíz y relaciones jerárquicas padre-hijo [1]. Esta abstracción matemática, formalizada desde el siglo XVIII, se traduce en programación como una estructura de datos: nodos enlazados por referencias o punteros que soportan operaciones eficientes de búsqueda, inserción y eliminación [1]. La transición no es trivial: mientras la teoría de grafos estudia propiedades abstractas (conectividad, ciclos, balance), la ingeniería de software prioriza la complejidad computacional (notación Big-O) y las restricciones físicas del hardware, como la memoria caché o el acceso a disco. De esa tensión entre teoría pura e implementación surgen estructuras especializadas —heaps, tries, árboles B, árboles de decisión— cada una optimizada para un problema concreto de programación, bases de datos o inteligencia artificial.

2. Núcleo de la investigación

A. Programación avanzada: Heaps, Tries y Árboles B

Es un árbol binario casi completo que cumple la propiedad de orden: cada nodo padre es mayor (max-heap) o menor (min-heap) que sus hijos [2]. Se implementa sobre un arreglo simple, evitando punteros explícitos. Su propósito de optimización clave es dar acceso $O(1)$ al elemento extremo y soportar inserción/extracción en $O(\log n)$, lo que lo convierte en la base de las colas de prioridad y del algoritmo Heapsort [2].

- Pros: acceso inmediato al máximo/mínimo; ordenamiento in-place sin memoria extra.
- Contras: buscar un elemento arbitrario cuesta $O(n)$; no mantiene el arreglo totalmente ordenado.
- Casos de uso: colas de prioridad, algoritmo de Dijkstra, planificadores de sistemas operativos.

Organiza claves —típicamente cadenas de texto— de modo que cada nodo representa un carácter y cada camino raíz-nodo un prefijo compartido [3]. A diferencia de un árbol binario de búsqueda, es un árbol n -ario cuyo factor de ramificación depende del tamaño del alfabeto y no del número de claves almacenadas.

- Pros: búsqueda en $O(m)$, con m = longitud de la clave, independiente del total de elementos; soporta búsquedas por prefijo de forma natural.
- Contras: alto consumo de memoria por los punteros de cada nodo (mitigable con tries comprimidos o radix trees).
- Casos de uso: autocompletado, correctores ortográficos, tablas de enrutamiento IP.

Generalizan el árbol binario de búsqueda permitiendo múltiples claves e hijos por nodo, y se mantienen siempre balanceados por construcción [4]. Su objetivo central es minimizar la altura del árbol y, con ella, el número de accesos a memoria secundaria, razón por la que dominan el diseño de índices de bases de datos [4], [5] (desarrollado en la sección B).

B. Bases de datos: índices B-Tree / B+-Tree

Un índice de base de datos es, en esencia, un árbol B o B+ que actúa como mapa ordenado entre valores de columna y ubicaciones físicas de los registros [4]. Al tratarse de un grafo acíclico balanceado con factor de ramificación k , la profundidad del árbol crece en $O(\log_k n)$, por lo que localizar un registro entre millones requiere solo unos pocos accesos a disco [5]. La diferencia entre B-Tree y B+-Tree es estructural: en un B+-Tree los datos residen únicamente en las hojas, y estas se enlazan secuencialmente entre sí, lo que optimiza tanto la búsqueda puntual como los recorridos por rango (ORDER BY, BETWEEN) [4], [6]. Motores como InnoDB (MySQL) o PostgreSQL emplean variantes de B+-Tree como estructura por defecto para sus índices [6].

Estructura	Datos en nodos internos	Búsqueda por rango	Complejidad de búsqueda	Caso de uso típico
B-Tree	Sí	Moderada	$O(\log n)$	Índices generales, sistemas de archivos
B+-Tree	No (solo en hojas enlazadas)	Excelente	$O(\log n)$	Índices de motores relacionales (MySQL, PostgreSQL)
Índice Hash	No aplica (tabla hash)	Nula	$O(1)$ promedio	Búsquedas de igualdad exacta (key-value stores)

C. Inteligencia artificial: árboles de decisión

Un árbol de decisión modela el aprendizaje supervisado como una secuencia de particiones jerárquicas del espacio de atributos: cada nodo interno aplica una prueba sobre una variable, cada rama representa un resultado posible y cada hoja asigna una clase o valor [7]. Algoritmos como ID3 y su sucesor C4.5 construyen el árbol de forma recursiva (top-down induction), seleccionando en cada nodo el atributo que maximiza la ganancia de información, es decir, la mayor reducción de entropía [7].

- Pros: alta interpretabilidad, ya que puede leerse como un conjunto de reglas si-entonces; no requiere normalizar los datos; admite variables categóricas y numéricas.
- Contras: tiende al sobreajuste (overfitting) en árboles profundos y presenta alta varianza ante pequeños cambios en los datos de entrenamiento.

Para reducir la varianza, métodos como Random Forest combinan cientos de árboles entrenados sobre subconjuntos aleatorios de datos y variables, promediando sus predicciones y mejorando la generalización respecto de un único árbol [8].

- Casos de uso: diagnóstico médico asistido, detección de fraude, sistemas de recomendación, motores de decisión en robótica.



Recursos TIC sugeridos para la exposición

La siguiente tabla muestra herramientas gratuitas o de bajo costo para representar visualmente las estructuras discutidas durante una presentación oral.

Herramienta	Tipo	Uso recomendado en la exposición
Graphviz	Motor de renderizado de grafos (lenguaje DOT)	Generar automáticamente diagramas de árboles B, tries o árboles de decisión a partir de código; útil para mostrar cómo cambia la estructura tras cada inserción.
VisuAlgo	Plataforma interactiva de visualización de algoritmos	Animar paso a paso operaciones sobre heaps, árboles B y tries (inserción, eliminación, rotaciones) en tiempo real durante la charla.
Lucidchart draw.io	/ Editor de diagramas colaborativo	Diseñar diagramas conceptuales manuales (comparativa B-Tree vs. B+-Tree, arquitectura de índices) para las diapositivas.
Python (scikit-learn + Matplotlib)	Bibliotecas de análisis y visualización de datos	Entrenar un árbol de decisión real y graficarlo con plot_tree sobre un dataset de ejemplo, mostrando el algoritmo en vivo.

FASE 4: Proyecto Integrador - Modelado y Optimización de Redes (Semana 16: 20-24 julio 2026)

a. Los equipos elaboran un informe técnico del proyecto que incluya: descripción del problema, modelado matemático (lógica, booleana, grafos), implementación en software, resultados obtenidos, análisis de optimización y conclusiones. (ARTICULO)

Informe Técnico: Exploración de Grafos de Conocimiento basada en Árboles Steiner de Grupo Cuadráticos (QGSTP) y el Algoritmo B3F

Descripción del problema: Cohesión semántica en KGs

La exploración de Grafos de Conocimiento (KGs) mediante consultas por palabras clave permite a usuarios sin experiencia técnica extraer información estructurada sin necesidad de dominar lenguajes de consulta formales [10]. Tradicionalmente, este proceso se ha modelado como el problema del Árbol Steiner de Grupo (GST), el cual busca un subgrafo conectado de peso mínimo que cubra todas las palabras clave de la consulta [10]. Sin embargo, los enfoques GST estándar presentan una limitación crítica al priorizar únicamente la relevancia individual mediante pesos de vértices o aristas, ignorando la relación semántica entre las entidades que conforman la respuesta [10]. Esta omisión suele producir resultados semánticamente incoherentes, donde las entidades carecen de un hilo conductor común [10]. La introducción del concepto de "cohesión" es fundamental para mitigar este fenómeno, garantizando que los subgrafos resultantes posean distancias semánticas reducidas entre sus componentes [10]. Un ejemplo ilustrativo se observa en la comparación entre T1 y T2 para la consulta "melvin schwartz, emil wolf": mientras T1 presenta una relación cohesiva centrada en físicos y sus mentores académicos, T2 ofrece una mezcla dispar de entidades que incluye instituciones y ciudades sin una conexión semántica directa de interés para la exploración [10].

Modelado matemático del problema QGSTP

Formalmente, un Grafo de Conocimiento se representa como una estructura dirigida $G = \langle V, E \rangle$, donde V es el conjunto de vértices y E el conjunto de aristas [10]. El problema QGSTP (Quadratic Group Steiner Tree Problem) extiende el modelo GST convencional mediante una función de costo que integra términos cuadráticos para cuantificar la cohesión [1]. El costo de un árbol $T = \langle V_T, E_T \rangle$ se define mediante la ecuación:

$$\text{cost}(T) = \alpha \sum \text{wt}(v) + (1 - \alpha) \sum \text{sd}(v_i, v_j) \quad [10]$$

En esta expresión, la segunda sumatoria se realiza estrictamente para pares donde $i < j$ para evitar la redundancia en el cálculo de distancias semánticas [10]. El parámetro $\alpha \in [0, 1]$ permite calibrar el equilibrio entre la relevancia (saliciencia) de los nodos, representada por $\text{wt}(v)$, y la cohesión semántica, representada por $\text{sd}(v_i, v_j)$ [10]. De acuerdo con el Teorema 1, el problema QGSTP es NP-hard [1]. La complejidad intrínseca del problema se demuestra mediante la reducción del problema de Árbol Steiner con pesos en vértices y el problema de cobertura de conjuntos cuadráticos [10]. Una dificultad adicional radica en que los valores de los pesos y las distancias semánticas suelen ser independientes entre sí, lo que impide optimizar ambos componentes de forma aislada [10].

Implementación: Algoritmo B3F (Branch-and-Bound Best-First)

El algoritmo B3F constituye el primer método exacto diseñado específicamente para resolver el problema QGSTP [10]. La exactitud de B3F se fundamenta matemáticamente en el Lemma 2, que establece que cualquier árbol T con un diámetro $\text{diam}(T) \leq 2d$ posee un radio $\text{rad}(T) \leq d$, garantizando la existencia de un vértice central (central vertex) situado a una distancia máxima d de todas las hojas [10]. Bajo esta premisa, B3F construye soluciones óptimas mediante la fusión de g caminos que convergen en dicho vértice común [10]. El proceso de ramificación (branching) consiste en la enumeración y expansión de caminos desde los vértices que contienen las palabras clave, donde cada camino p crece una arista a la vez ($p \text{ grow ptop to } v \text{ with one edge}$) [10]. Para gestionar el espacio de búsqueda, el algoritmo emplea una estrategia Best-First utilizando una cola de prioridad que procesa primero los caminos con el menor costo estimado [10]. Este enfoque permite explorar el espacio de forma descendente, garantizando que la primera solución completa que supere el límite inferior de los elementos en la cola sea la óptima [10].

Estrategias de Optimización: Estimación de Límites Inferiores

La eficiencia computacional de B3F radica en su capacidad de poda mediante la estimación de límites inferiores para evitar ramas subóptimas [10]. Para el componente de peso (cost_wt), el algoritmo resuelve una instancia del Problema de Cobertura de Conjuntos Pesados (WSCP) para determinar el peso mínimo necesario para cubrir las palabras clave restantes de la consulta [10]. Para el componente de cohesión (cost_sd), se

aplica una combinación del Problema de Cobertura de Conjuntos (SCP) y la teoría de emparejamiento máximo (Maximum Matching) [10]. En este proceso, se construye un grafo completo H sobre el conjunto de vértices del camino actual, donde el peso de las aristas es la distancia semántica [10]. Dado que la distancia semántica cumple con la desigualdad triangular, el peso del emparejamiento máximo M en H constituye un límite inferior válido para la cohesión total de cualquier árbol que contenga dicho camino [10]. El uso de SCP permite determinar el número mínimo de vértices adicionales necesarios, integrando esta cifra en la estimación final para asegurar que $\text{cost}(p) \leq \min \text{cost}(T)$ en todo momento [10].

Resultados Experimentales y Evaluación de Rendimiento

La evaluación de B3F se realizó sobre cinco versiones de tres KGs públicos, cubriendo diversas escalas y densidades de datos [10].

Tabla 1: Resumen de KGs y configuración experimental

KG	Fuente	$ V $	$ E $	Consultas	g (Keywords)
MND	MONDIAL	40,890	120,690	34	2–4
L1	LUBM	20,797	82,590	200	2–6
L4	LUBM	91,045	370,654	200	2–6
D20K	DBpedia	20,000	37,017	128	2–6
D100K	DBpedia	100,000	205,743	241	2–6

[10].

Tabla 2: Comparativa de Rendimiento y Ratio de Cohesión

Métrica	B3F ($\alpha = 0.3$)	B3F ($\alpha = 0.7$)	DPBF (GST)
Mean Run Time (L4)	30.96 s	32.50 s	113.00 s
Mean Run Time (D100K)	261.14 s	101.12 s	25.41 s
Cohesiveness Ratio (MND)	0.89	0.92	1.00
Cohesiveness Ratio (L4)	0.53	0.63	1.00

[10].

El análisis de los datos indica que B3F produce respuestas entre un 8% y un 53% más cohesivas que el algoritmo DPBF, según el ratio de cohesión ($\text{cost_sd}(T_B3F) / \text{cost_sd}(T_DPBF)$), donde valores inferiores a 1.0 representan una mejora en la calidad semántica [10]. En términos de eficiencia, B3F demuestra ser competitivo en grafos de tamaño medio; sin embargo, en datasets extensos como D100K, el tiempo de ejecución presenta un crecimiento superlineal respecto al número de vértices [10]. Esta observación es crítica, ya que confirma que si bien B3F es ideal para aplicaciones que priorizan la calidad de la respuesta en KGs empresariales, los escenarios de escala masiva podrían requerir algoritmos de aproximación [10].

Conclusion

Este informe valida la eficacia del modelo QGSTP para resolver la falta de cohesión semántica en la exploración de Grafos de Conocimiento [10]. El algoritmo B3F se consolida como la primera solución exacta para este problema, integrando técnicas avanzadas de optimización combinatoria para manejar la complejidad NP-hard del modelo cuadrático [10]. La capacidad de B3F para mejorar significativamente la cohesión de las respuestas en tiempos comparables a los métodos GST estándar lo posiciona como una base robusta para el desarrollo de futuras herramientas de búsqueda semántica y algoritmos de aproximación escalables [10].



unl

Universidad
Nacional
de Loja

6. Preguntas de Control

6.1 Explique la diferencia entre los algoritmos BFS y DFS. ¿En qué situaciones sería más apropiado utilizar BFS y en cuáles DFS? Argumente con ejemplos de aplicaciones reales en la ciencia de la computación

¿Qué es BFS (Breadth-First Search)?

BFS, o búsqueda en anchura, es un algoritmo de recorrido de grafos y árboles que explora el nodo raíz primero y luego visita todos los nodos vecinos de un mismo nivel antes de avanzar al siguiente nivel. Utiliza una estructura de datos tipo cola (FIFO: primero en entrar, primero en salir) para gestionar el orden de visita de los nodos [1].

¿Cuándo usar BFS?

- Camino más corto: en grafos no ponderados, BFS garantiza encontrar la ruta con menor número de saltos entre dos nodos.
- Redes sociales: para calcular el 'grado de separación' entre dos personas (por ejemplo, sugerencias de amistad en Facebook o LinkedIn).
- Enrutamiento de redes: protocolos que necesitan explorar la red por niveles de cercanía, como el descubrimiento de vecinos.
- Web crawlers: rastreadores que exploran un sitio web nivel por nivel a partir de una página inicial.
- Redes P2P: búsqueda de recursos cercanos antes de explorar nodos más lejanos.

¿Qué es DFS (Depth-First Search)?

DFS, o búsqueda en profundidad, explora un camino del grafo tan lejos como sea posible antes de retroceder (backtracking) para explorar otras ramas. Utiliza una estructura de datos tipo pila (LIFO: último en entrar, primero en salir), ya sea de forma explícita o mediante la pila de llamadas de la recursión [1], [2].

¿Cuándo usar DFS?

- Detección de ciclos: es más natural con DFS, útil en compiladores para detectar dependencias circulares.
- Ordenamiento topológico: usado en sistemas de compilación (como Make) y gestores de paquetes para resolver el orden de instalación de dependencias.
- Resolución de laberintos y puzzles: como el Sudoku o el problema de las N-reinas, mediante backtracking.
- Componentes conexos y fuertemente conexos: algoritmos como el de Tarjan y Kosaraju se basan en DFS.
- Recorrido de sistemas de archivos: explorar carpetas y subcarpetas en profundidad antes de pasar a la siguiente carpeta del mismo nivel.

Comparación general

Criterio	BFS	DFS
Estructura de datos	Cola (FIFO)	Pila (LIFO) o recursión
Estrategia	Explora nivel por nivel (por anchura)	Explora una rama completa antes de retroceder
Memoria utilizada	Puede ser alta: guarda todos los nodos de un nivel	Generalmente menor: solo guarda el camino actual
Encuentra el camino más corto (grafo no ponderado)	Sí, garantizado	No garantizado
Implementación típica	Iterativa con cola	Recursiva o iterativa con pila



7. Conclusiones

Se concluye que:

- Se logró comprender y dominar la terminología esencial de la teoría de grafos (vértices, aristas, grados, adyacencia e incidencia) mediante la modelación de situaciones del lenguaje natural a estructuras discretas. Asimismo, la representación de un mismo grafo a través de diagramas visuales, matrices de adyacencia y listas de adyacencia permitió contrastar la eficiencia computacional de cada método, concluyendo que la elección de la estructura de datos impacta directamente en la optimización de la memoria y la velocidad de procesamiento según la densidad del grafo.
- Se concluye que el software Gephi demostró ser una herramienta analítica y de visualización indispensable para la ingeniería, al permitir transformar la abstracción matemática de la teoría de grafos en un modelo digital. A través de su interfaz, se logró calcular con precisión métricas estructurales críticas de conectividad y centralidad, comprobando que el uso de herramientas computacionales de modelado es clave para diagnosticar fallos en redes físicas, optimizar recursos logísticos y diseñar soluciones tecnológicas aplicadas a problemas de infraestructura del mundo real.

8. Reflexiones críticas personales

Andrade Erick

Al iniciar este trabajo, las matrices y el algoritmo de Dijkstra fueron la parte que más me costó entender: sonaban complicados solo de leer sus nombres. Sin embargo, lo que más me sorprendió fue descubrir que, una vez que entiendes la lógica detrás de cada paso, todo se vuelve mucho más intuitivo de lo que parecía al principio; no era tan difícil como yo pensaba, solo requería seguir el razonamiento con calma. Esta experiencia se conecta directamente con mi interés profesional por la inteligencia artificial, un campo donde estructuras como los grafos y algoritmos de optimización como Dijkstra son la base de muchos sistemas: desde motores de búsqueda hasta modelos que deben encontrar la mejor decisión posible entre miles de opciones. Este trabajo me confirmó que entender bien los fundamentos, aunque al inicio parezcan intimidantes, es lo que después permite avanzar con seguridad hacia temas más avanzados como la IA.

Tyrone Encarnación

El desarrollo de este trabajo me permitió ver que la teoría de grafos va más allá de la teoría matemática, siendo un método muy eficiente para representar y solucionar problemas complejos de la vida real. Al mapear el sistema vial de Loja, comprobé cómo elementos clave como los nodos y enlaces organizan la lógica urbana, haciendo que su evaluación técnica sea mucho más sencilla.

Por otra parte, analizar e implementar paso a paso las búsquedas BFS y DFS, además del algoritmo de Dijkstra, fue clave para comprender su funcionamiento interno. Este ejercicio me ayudó a visualizar claramente el rol de las colas y pilas al rastrear los vértices, y cómo se calculan las rutas óptimas. Comparar estos cálculos manuales con el sitio Graph Online no solo validó mis resultados, sino que me hizo valorar la importancia del software especializado para procesar redes a gran escala.

Matías Ortega

Durante esta práctica pude comprender que la teoría de grafos no es solo un conjunto de conceptos matemáticos, sino una herramienta muy útil para representar y resolver problemas reales. Al modelar redes como la de transporte de la ciudad de Loja, identifiqué



cómo los vértices, las aristas y sus propiedades permiten describir situaciones cotidianas de forma estructurada y facilitar su análisis.

La ejecución manual de los algoritmos BFS, DFS y Dijkstra me ayudó a entender el funcionamiento interno de cada uno, observando paso a paso cómo se recorren los nodos, cómo se administran las estructuras de cola y pila, y cómo se calculan las rutas más cortas. Posteriormente, al comparar estos resultados con los obtenidos en Graph Online, pude verificar la precisión de los procedimientos y comprender la importancia del software como apoyo para analizar grafos más complejos.

En lo personal, considero que esta práctica fortaleció mi razonamiento lógico y mi capacidad para resolver problemas de manera organizada. También me permitió reconocer la aplicación de la teoría de grafos en áreas como las redes de transporte, las telecomunicaciones, la informática y la optimización de procesos. Sin duda, estos conocimientos serán de gran utilidad en mi formación como ingeniero, ya que constituyen una base importante para el diseño y análisis de sistemas computacionales.

Jhoan Rojas

El estudio práctico de BFS y DFS, en combinación con el algoritmo de Dijkstra, me ha permitido entender que la teoría de grafos no se limita a recorrer estructuras, sino a modelar la toma de decisiones bajo restricciones reales. Mientras que BFS y DFS ofrecen estrategias fundamentales para explorar redes según la cercanía por niveles o la profundidad de los caminos, Dijkstra eleva este análisis al introducir el concepto de peso o costo en las aristas, transformando una búsqueda topológica en un problema de optimización logística y económica. Comprender esta transición entre algoritmos que miden "saltos" y aquellos que evalúan "costos reales" es indispensable para diseñar soluciones eficientes en el mundo físico, como optimizar rutas de transporte o planificar redes de conectividad que respondan a las verdaderas necesidades geográficas de una región.

9. Bibliografía / Referencias

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022. ISBN: 978-0-262-04630-5. [Online]. Available: <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- [2] Oscar Levin, University of Northern Colorado (2022). *Discrete Mathematics: An Open Introduction – 4th Edition*. Universidad Tecnológica de Pereira. ISBN: 9781534970748.
- [3] J. W. J. Williams, "Algorithm 232: Heapsort," *Commun. ACM*, vol. 7, no. 6, pp. 347–348, Jun. 1964, doi: 10.1145/512274.3734138. [Online]. Available: <https://doi.org/10.1145/512274.3734138>
- [4] E. Fredkin, "Trie memory," *Commun. ACM*, vol. 3, no. 9, pp. 490–499, Sep. 1960, doi: 10.1145/367390.367400. [Online]. Available: <https://doi.org/10.1145/367390.367400>
- [5] R. Bayer and E. M. McCreight, "Organization and maintenance of large ordered indexes," *Acta Inform.*, vol. 1, no. 3, pp. 173–189, 1972, doi: 10.1007/BF00288683. [Online]. Available: <https://doi.org/10.1007/BF00288683>
- [6] D. Comer, "The ubiquitous B-tree," *ACM Comput. Surv.*, vol. 11, no. 2, pp. 121–137, Jun. 1979, doi: 10.1145/356770.356776. [Online]. Available: <https://doi.org/10.1145/356770.356776>
- [7] G. Graefe, "Modern B-tree techniques," *Found. Trends Databases*, vol. 3, no. 4, pp. 203–402, 2011, doi: 10.1561/19000000028. [Online]. Available: <https://doi.org/10.1561/19000000028>
- [8] J. R. Quinlan, "Induction of decision trees," *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, 1986, doi: 10.1007/BF00116251. [Online]. Available: <https://doi.org/10.1007/BF00116251>
- [9] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324. [Online]. Available: <https://doi.org/10.1023/A:1010933404324>



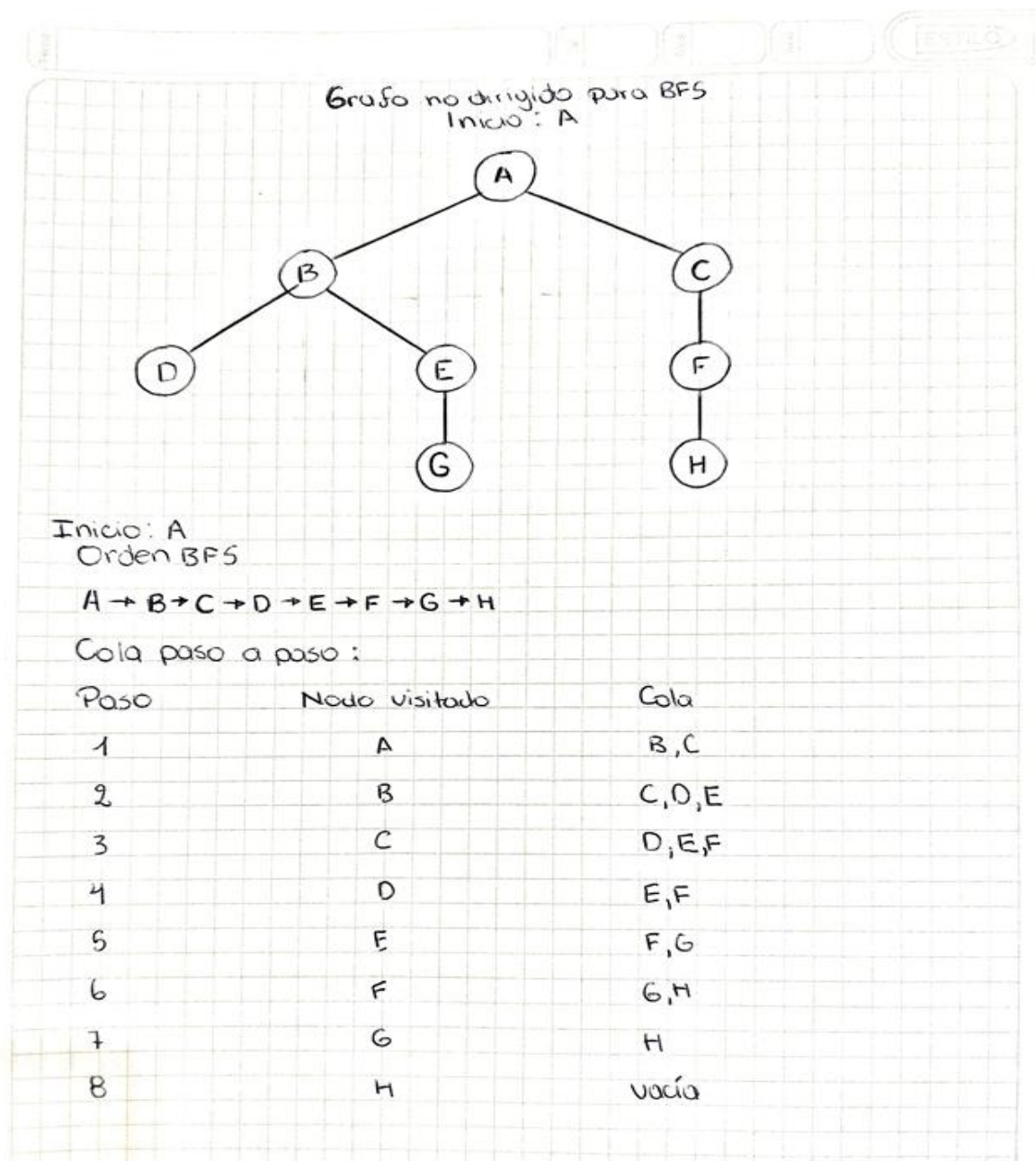
- [10] Y. Shi, G. Cheng, T. K. Tran, J. Tang and E. Kharlamov, "Keyword-Based Knowledge Graph Exploration Based on Quadratic Group Steiner Trees," in Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI 2021), 2021, pp. 1555-1562. doi: 10.24963/ijcai.2021/215.

10. Anexos

FASE 2, PUNTO 2: Los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra)

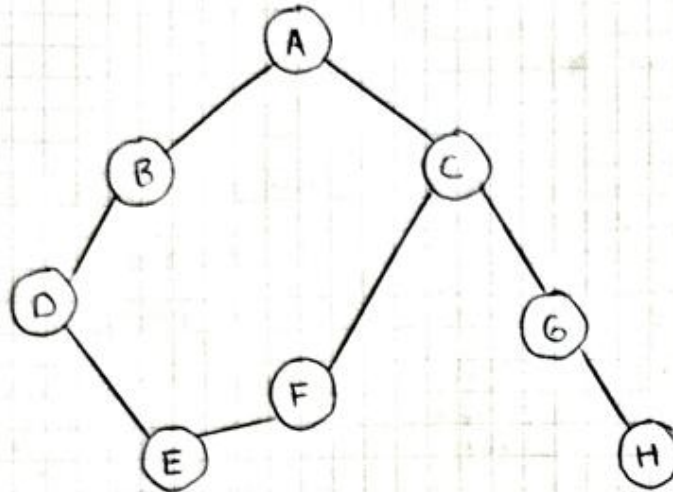
Jhoan Rojas

Algoritmo BFS



Algoritmo DFS

Grafo no dirigido para DFS
Inicio: A



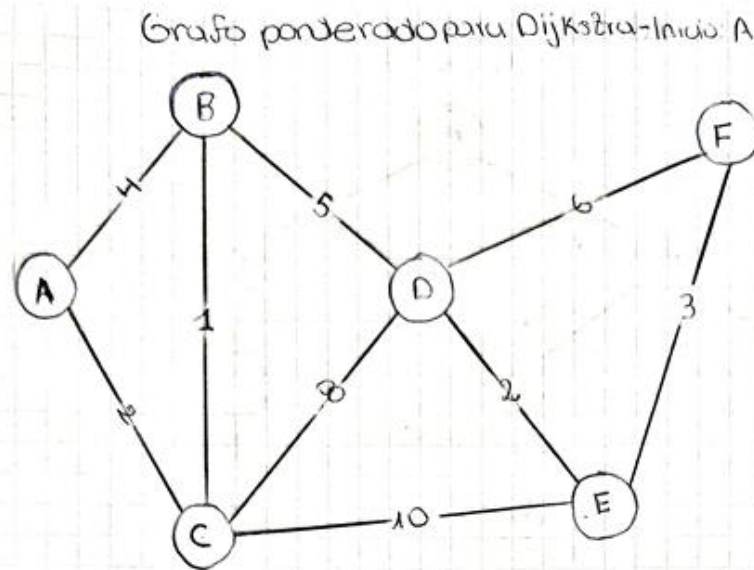
Inicio: A
Orden DFS

A → B → D → E → F → C → G → H

Pila / recorrido:

Paso	Nodo visitado	Acción
1	A	va a B
2	B	va a D
3	D	va a E
4	E	va a F
5	F	va a C
6	C	va a G
7	G	va a H
8	H	termina

Algoritmo Dijkstra



Inicio: A

Distancias Finales desde A:

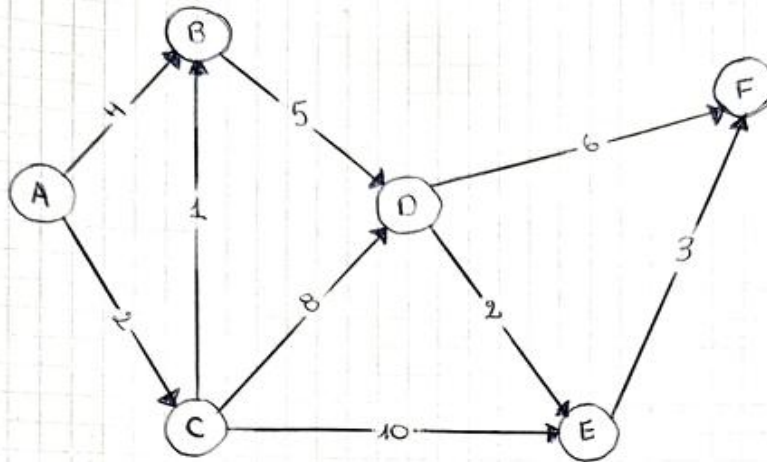
Nodo	Distancia mínima	Camino
A	0	A
C	2	A → C
B	3	A → C → B
D	8	A → C → B → D
E	10	A → C → B → D → E
F	13	A → C → B → D → E → F

Camino más corto de A a F:

$$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F = 13$$

Algoritmo Dijkstra

Grupo ponderado dirigido para Dijkstra
Nodo Inicial : A



Dijkstra paso a paso

Paso	Nodo Fijo	Distancias acumuladas
1	A	$A=0, B=4, C=2, D=\infty, E=\infty, F=\infty$
2	C	$A=0, B=3, C=2, D=10, E=12, F=\infty$
3	B	$A=0, B=3, C=2, D=8, E=12, F=\infty$
4	D	$A=0, B=3, C=2, D=8, E=10, F=14$
5	E	$A=0, B=3, C=2, D=8, E=10, F=13$
6	F	$A=0, B=3, C=2, D=8, E=10, F=13$

Orden de descubrimiento / fijación

$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$

Camino resultantes

Nodo destino

Distancia mínima

Camino

A

0

A

C

2

A → C

B

3

A → C → B

D

8

A → C → B → D

E

10

A → C → B → D → E

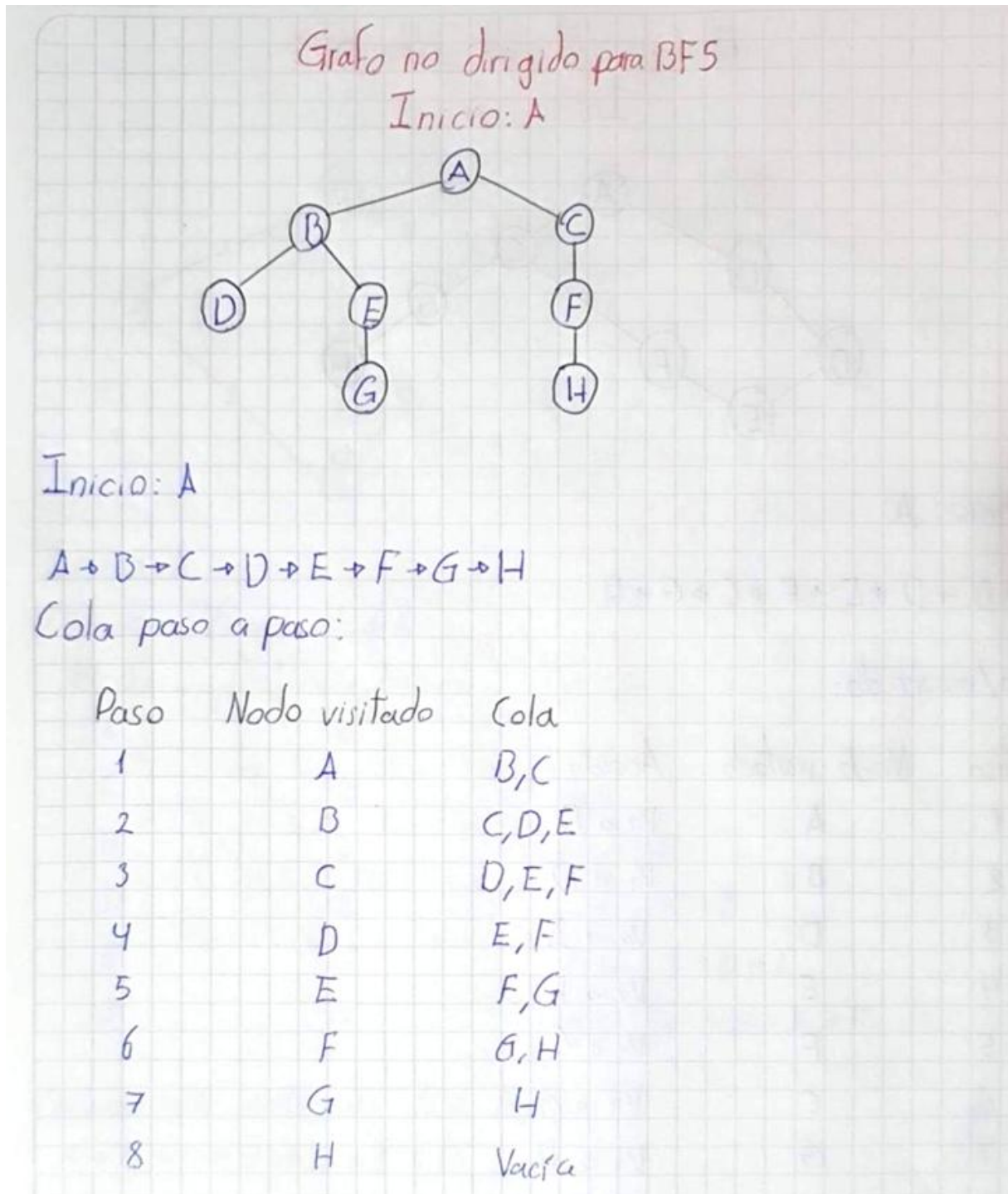
F

13

A → C → B → D → E → F

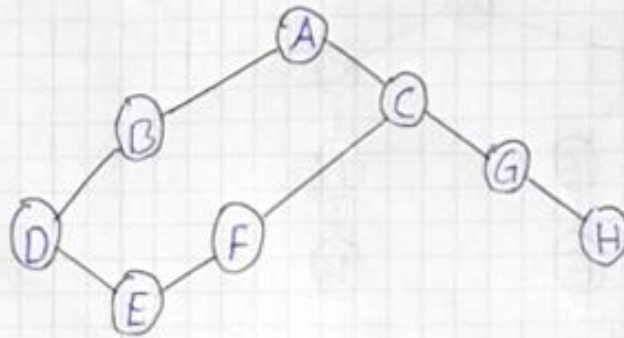
Camino más corto de A a F:

A → C → B → D → E → F = 13

Erick Andrade**Algoritmo BFS**

Algoritmo DFS

Grafo no dirigido para DFS
Inicio: A



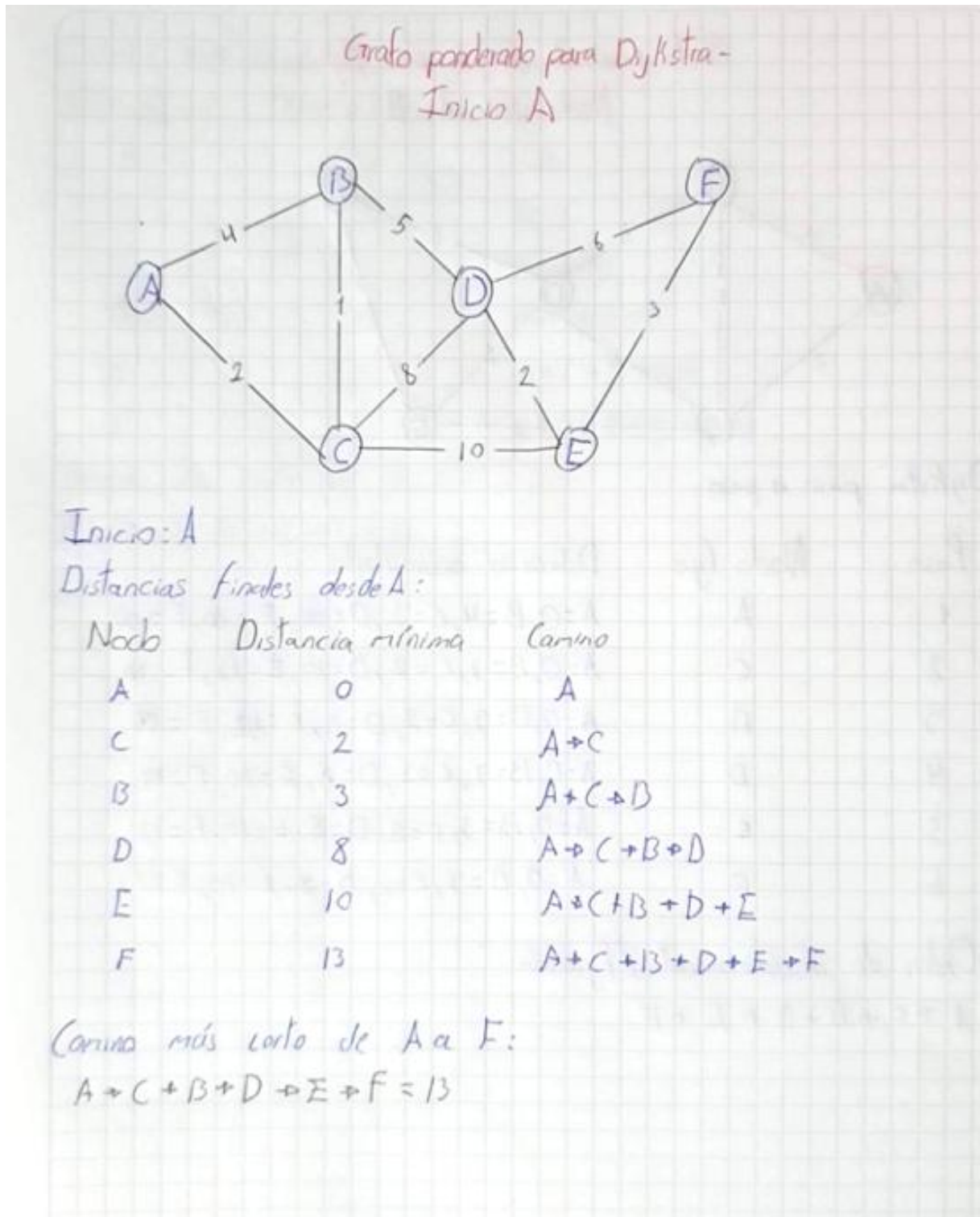
Inicio: A

A → B → D → E → F → C → G → H

Pila/recorrido:

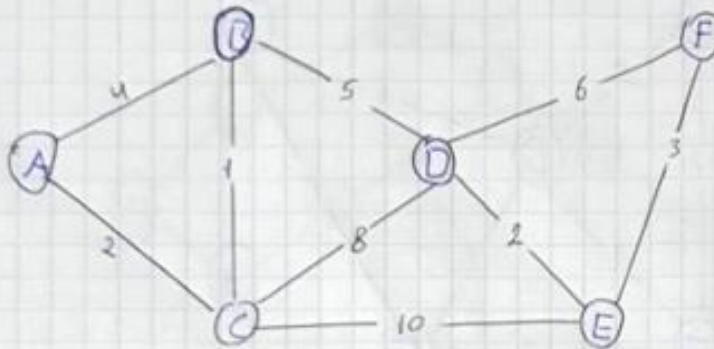
Paso	Nodo visitado	Acción
1	A	Va a B
2	B	Va a D
3	D	Va a E
4	E	Va a F
5	F	Va a C
6	C	Va a G
7	G	Va a H
8	H	Termina

Algoritmo Dijkstra



Algoritmo Dijkstra

Grato ponderado dirigido para Dijkstra
Nodo inicial: A



Dijkstra paso a paso

Paso	Nodo fijo	Distancias acumuladas
1	A	$A=0, B=4, C=2, D=\infty, E=\infty, F=\infty$
2	C	$A=0, B=3, C=2, D=10, E=12, F=\infty$
3	B	$A=0, B=3, C=2, D=8, E=12, F=\infty$
4	D	$A=0, B=3, C=2, D=8, E=10, F=14$
5	E	$A=0, B=3, C=2, D=8, E=10, F=13$
6	F	$A=0, B=3, C=2, D=8, E=10, F=13$

Orden de descubrimiento / fijación

$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$

Caminos resultantes

Nodo destino	Distancia mínima	Camino
A	0	A
C	2	A → C
B	3	A → C → B
D	8	A → C → B → D
E	10	A → C → B → D → E
F	13	A → C → B → D → E → F

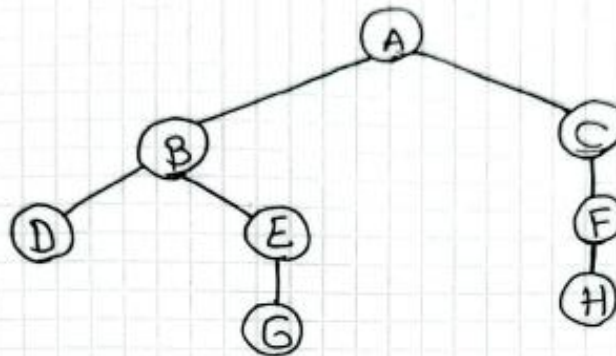
Camino más corto de A a F:

$$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F = 13$$

Tyrone Encarnación

Algoritmo BFS

Grafo no dirigido para BFS
Inicio : A.



Inicio : A

Orden BFS

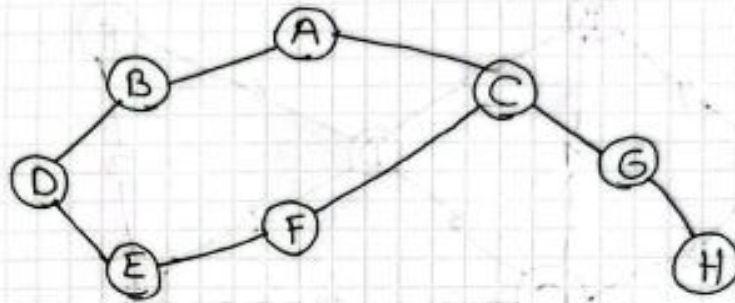
$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H$

Cola paso a paso

Paso	Nodo visitado	Cola
1	A	B, C
2	B	C, D, E
3	C	D, E, F
4	D	E, F
5	E	F, G
6	F	G, H
7	G	H
8	H	Vacía

Algoritmo DFS

Grafo no dirigido para DFS
Inicio A



Inicio A:

orden DFS:

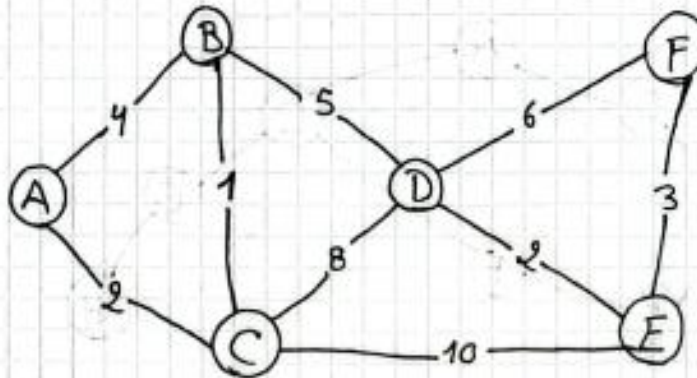
A → B → D → E → F → C → G → H

Pila / recorrido

Paso	Nodo visitado	Acción
1	A	Va a B
2	B	va a D
3	D	va a E
4	E	va a F
5	F	va a C
6	C	va a G
7	G	va a H
8	H	termina

Algoritmo Dijkstra

Grafo ponderado para Dijkstra
Inicio: A.



Inicio A
Distancias finales desde A:

Nodo	Distancia mínima	Camino
A	0	A
C	2	A → C
B	3	A → C → B
D	8	A → C → B → D
E	10	A → C → B → D → E
F	13	A → C → B → D → E → F

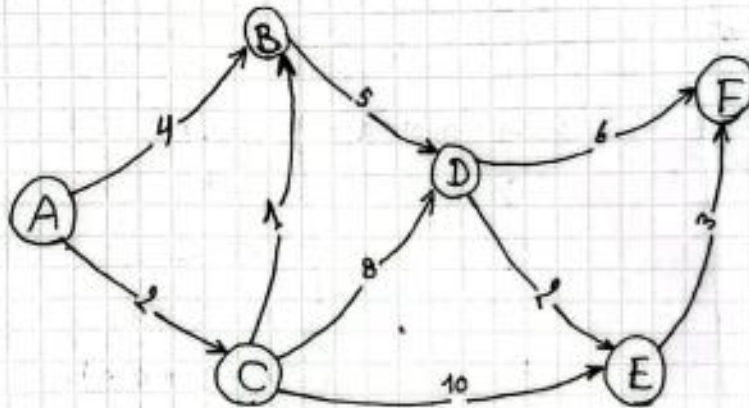
Camino mas corto de A a F:

A → C → B → D → E → F = 13

Algoritmo Dijkstra

Grafo ponderado dirigido para Dijkstra

Nodo inicial: A



Dijkstra paso a paso

Paso	Nodo fijo	Distancias acumuladas
1	A	$A=0, B=4, C=2, D=\infty, E=\infty, F=\infty$
2	C	$A=0, B=3, C=2, D=10, E=12, F=\infty$
3	B	$A=0, B=3, C=2, D=8, E=12, F=\infty$
4	D	$A=0, B=3, C=2, D=8, E=10, F=14$
5	E	$A=0, B=3, C=2, D=8, E=10, F=13$
6	F	$A=0, B=3, C=2, D=8, E=10, F=13$

Orden de descubrimiento/fijación

$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$

Camino resultantes / fijación

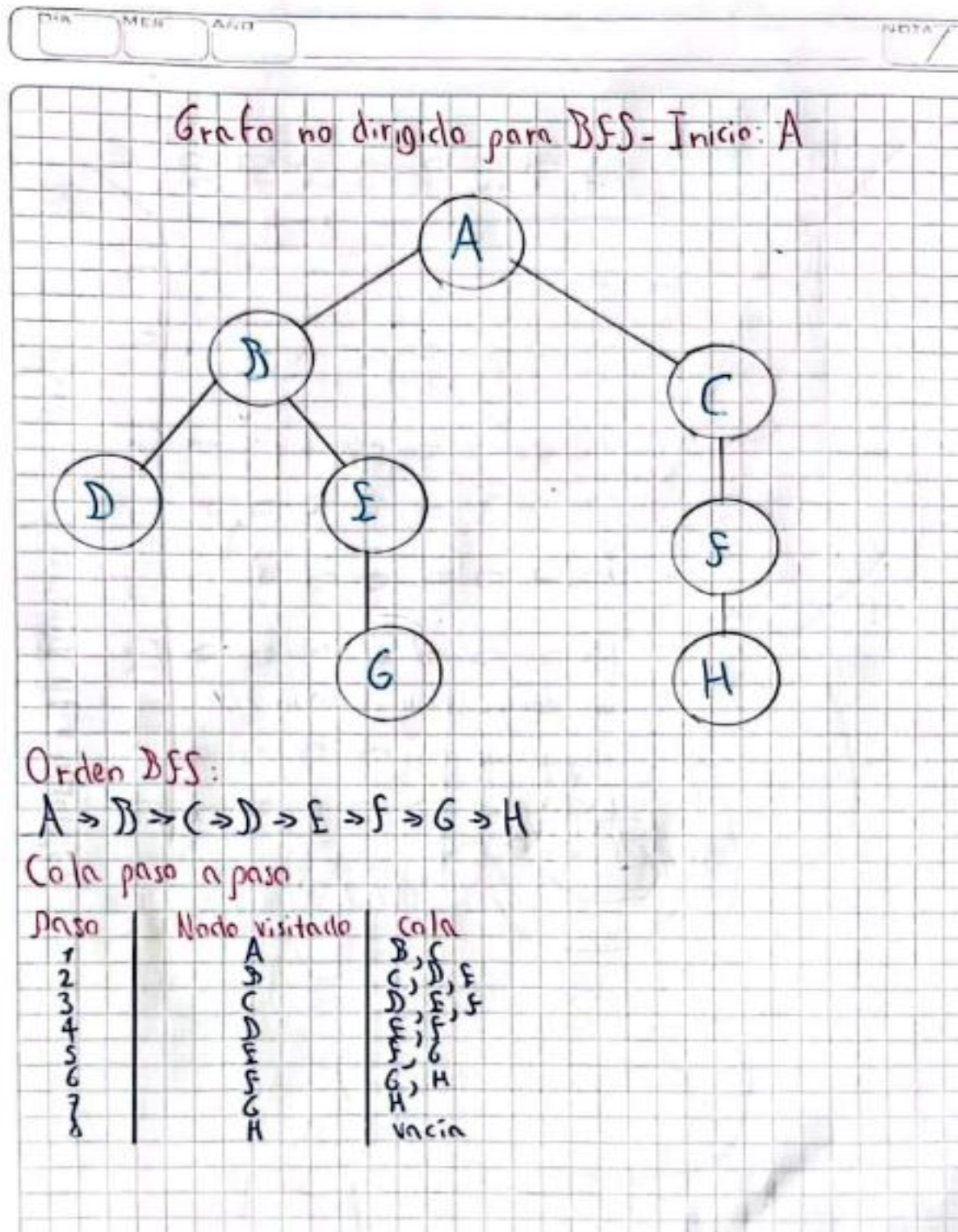
$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$

Camino resultantes

Nodo destino	Distancia mínima	camino
A	0	A
C	2	$A \rightarrow C$
B	3	$A \rightarrow C \rightarrow B$
D	8	$A \rightarrow C \rightarrow B \rightarrow D$
E	10	$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E$
F	13	$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$

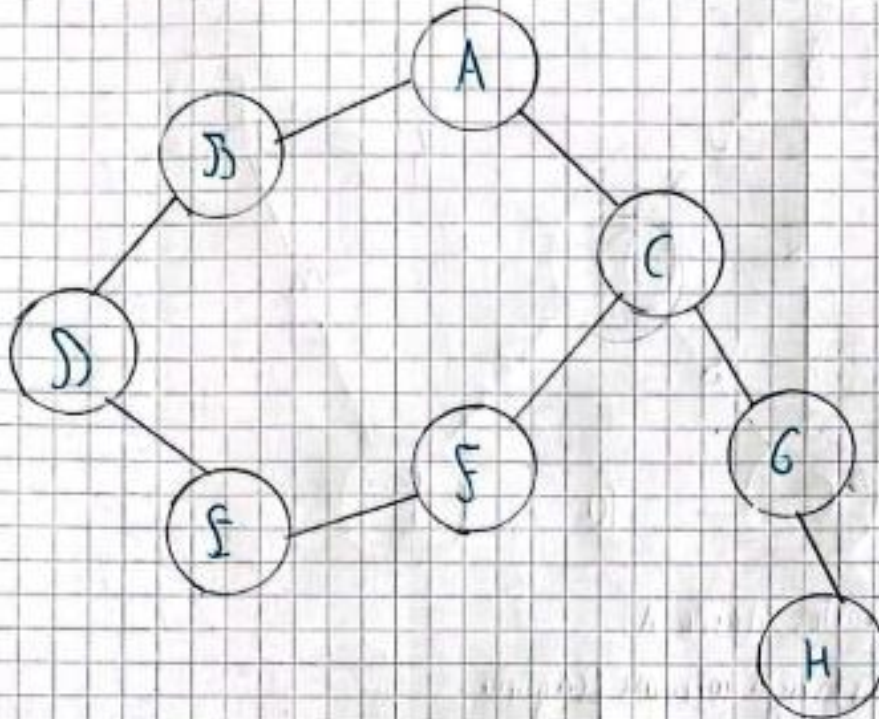
Matías Ortega

Algoritmo BFS



Algoritmo DFS

Grafo no dirigido para DFS - Inicio: A



Orden DFS:

A → B → D → E → F → C → G → H

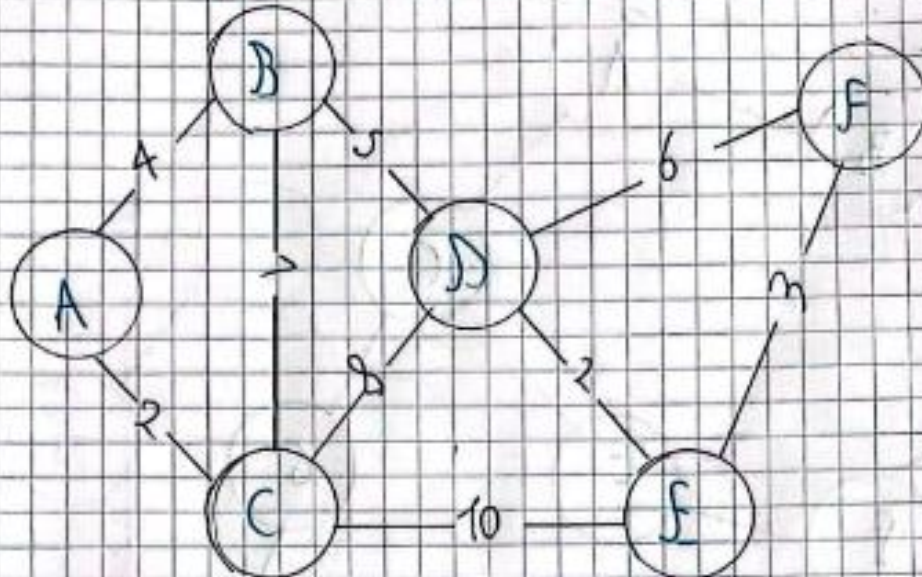
Pila / recorrido:

Paso	Nodo visitado	Acción
1	A	va a B
2	B	va a D
3	D	va a E
4	E	va a F
5	F	va a C
6	C	va a G
7	G	va a H
8	H	termina

Algoritmo Dijkstra

DIA MES AÑO NOTA

Grafo ponderado para Dijkstra - Inicio: A



Distancias Finales desde A:

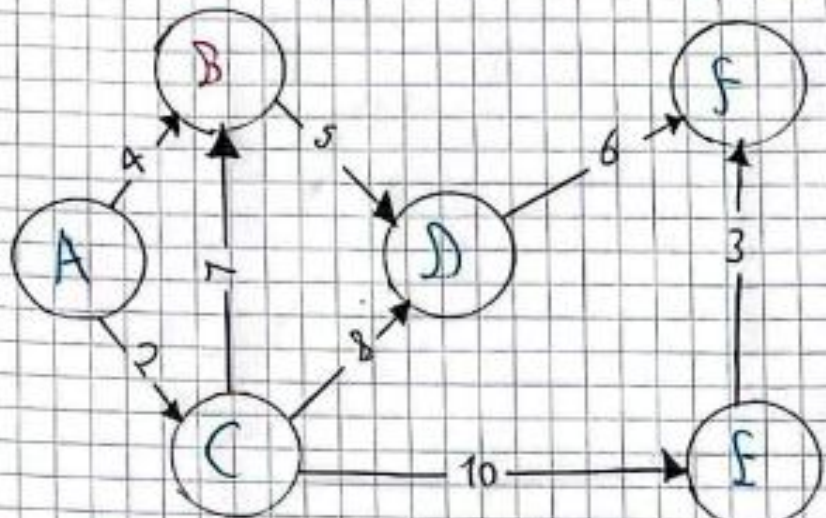
Nodo	Distancia mínima	Camino
A	0	A
C	2	A → C
B	3	A → C → B
D	8	A → C → B → D
E	10	A → C → D → E
F	13	A → C → B → D → E → F

Camino más corto de A a F:

A → C → B → D → E → F = 13

Algoritmo Dijkstra

Grafo ponderado dirigido para Dijkstra - Nodo inicial: A



Dijkstra paso a paso

Paso	Nodo fijo	Distancias acumuladas
1	A	A=0, B=4, C=2, D=∞, E=∞, F=∞
2	C	A=0, B=3, C=2, D=10, E=12, F=∞
3	B	A=0, B=3, C=2, D=8, E=12, F=∞
4	D	A=0, B=3, C=2, D=8, E=10, F=14
5	E	A=0, B=3, C=2, D=8, E=10, F=13
6	F	A=0, B=3, C=2, D=8, E=10, F=13

Orden de descubrimiento / fijación

A → C → B → D → E → F

Camino resultante	Distancia mínima	Camino
A	0	A
C	2	A → C
B	3	A → C → B
D	8	A → C → B → D
E	10	A → C → B → D → E
F	13	A → C → B → D → E → F

Camino más corto de A a F:
A → C → B → D → E → F = 13